EX.NO.1	CNN MODEL USING TENSOR FLOW
DATE: 09.07.26	

AIM:

To implement a basic Convolutional Neural Network (CNN) using TensorFlow/Keras for recognizing handwritten digits from the MNIST dataset and evaluate its classification accuracy.

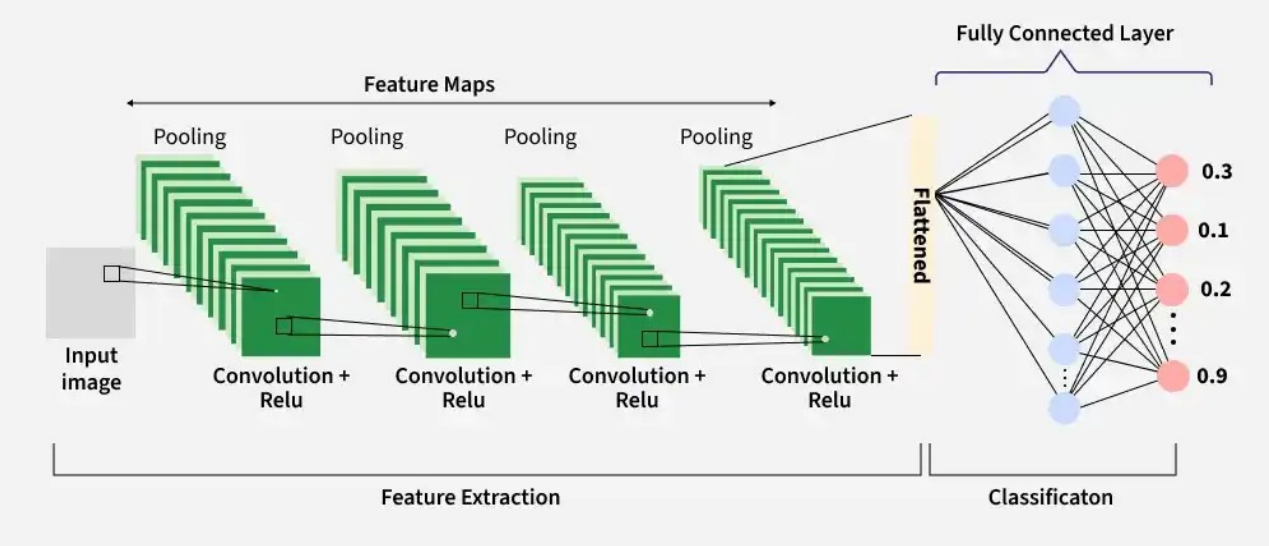
DESCRIPTION:

Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a type of deep learning algorithm specifically designed for image processing and computer vision applications. Unlike tr
aditional neural networks, CNNs automatically learn important image features such as edges, textures, patterns, and shapes using convolution operations. These extracted features are then used to classify or recognize images with high accuracy. CNNs are widely used in applications such as handwritten digit recognition, face recognition, object detection, medical image analysis, autonomous vehicles, and biometric authentication.

CNN ARCHITECTURE:

The CNN architecture used in this experiment consists of the following layers:



WORKFLOW OF CNN ARCHITECTURE:

Input Image → Convolution + ReLU → Max Pooling → Convolution + ReLU → Max Pooling →

Flatten → Fully Connected (Dense) Layer → Softmax Output Layer → Predicted Digit (0–9)

IMPORT LIBRARIES:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
```

```
import matplotlib.pyplot as plt
import numpy as np
```

PROGRAM:

Mount Google Drive:

```
from google.colab import drive
print("STEP 1 : MOUNTING GOOGLE DRIVE")
drive.mount('/content/drive')
print("Google Drive Mounted Successfully.")
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

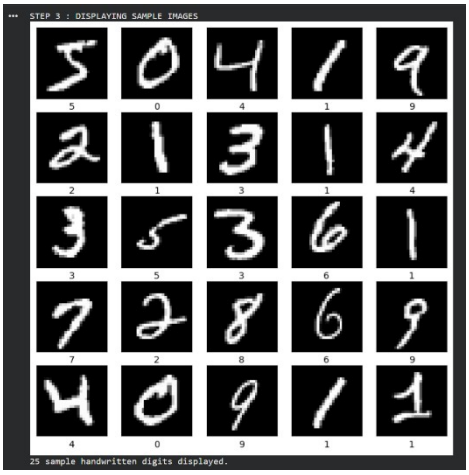
Load Dataset:

```
print("STEP 2 : LOADING MNIST DATASET")
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
print("Training Images :", train_images.shape)
print("Training Labels :", train_labels.shape)
print("Testing Images  :", test_images.shape)
print("Testing Labels  :", test_labels.shape)
print("\nNormalizing pixel values...")
train_images = train_images / 255.0
test_images = test_images / 255.0
print("Normalization Completed")
print("\nReshaping images to (28,28,1)...")
train_images = train_images.reshape((60000,28,28,1))
test_images = test_images.reshape((10000,28,28,1))
print("Training Shape :", train_images.shape)
print("Testing Shape  :", test_images.shape)
```

```
Dataset Loaded Successfully
Training Images : (60000, 28, 28)
Training Labels : (60000,)
Testing Images  : (10000, 28, 28)
Testing Labels  : (10000,)
```

Verify the Dataset:

```
print("STEP 3 : DISPLAYING SAMPLE IMAGES")
plt.figure(figsize=(8,8))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i].reshape(28,28), cmap='gray')
    plt.xlabel(train_labels[i])
plt.show()
print("25 sample handwritten digits displayed.")
```



Create the Convolutional Base:

```
print("STEP 4 : BUILDING CNN MODEL")
model = models.Sequential()
print("Adding Input Layer")
model.add(layers.Input(shape=(28,28,1)))
print("Adding First Convolution Layer")
model.add(layers.Conv2D(32,(3,3),activation='relu'))
print("Adding First MaxPooling Layer")
model.add(layers.MaxPooling2D((2,2)))
print("Adding Second Convolution Layer")
model.add(layers.Conv2D(64,(3,3),activation='relu'))
print("Adding Second MaxPooling Layer")
model.add(layers.MaxPooling2D((2,2)))
print("\nConvolutional Base Created Successfully.")
```

```
============================================================
STEP 4 : BUILDING CNN MODEL
============================================================
Adding Input Layer
Adding First Convolution Layer
Adding First MaxPooling Layer
Adding Second Convolution Layer
Adding Second MaxPooling Layer

Convolutional Base Created Successfully.
```

Display the Architecture:

```
print("STEP 5 : CNN ARCHITECTURE")
model.summary()
```

```
STEP 5 : CNN ARCHITECTURE
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0

```
Total params: 18,816 (73.50 KB)
Trainable params: 18,816 (73.50 KB)
Non-trainable params: 0 (0.00 B)
```

Add Dense Layers:

```
print("STEP 6 : ADDING DENSE LAYERS")
model.add(layers.Flatten())
print("Flatten Layer Added")
model.add(layers.Dense(64,activation='relu'))
print("Dense Layer (64 Neurons) Added")
model.add(layers.Dense(10,activation='softmax'))
print("Output Layer (10 Neurons) Added")
print("\nComplete CNN Model Created.")
```

```
STEP 6 : ADDING DENSE LAYERS
Flatten Layer Added
Dense Layer (64 Neurons) Added
Output Layer (10 Neurons) Added

Complete CNN Model Created.
```

Display Complete Model:

```
print("STEP 7 : COMPLETE MODEL SUMMARY")
model.summary()
```

```
================================================================
STEP 7 : COMPLETE MODEL SUMMARY
================================================================
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

```
Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)
```

Compile and Train the Model:

```
print("STEP 8 : COMPILING MODEL")
model.compile(
```

```
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
print("Optimizer : Adam")
print("Loss      : Sparse Categorical Crossentropy")
print("Metric    : Accuracy")
print("\nTraining Started...\n")
history = model.fit(
    train_images,
    train_labels,
    epochs=5,
    validation_data=(test_images,test_labels)
)
print("\nTraining Completed Successfully.")
```

```
STEP 8 : COMPILING MODEL
Optimizer : Adam
Loss      : Sparse Categorical Crossentropy
Metric    : Accuracy

Training Started...

Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 52s 27ms/step - accuracy: 0.9445 - loss: 0.2691 - val_accuracy: 0.9797 - val_loss: 0.0742
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 52s 28ms/step - accuracy: 0.9810 - loss: 0.0641 - val_accuracy: 0.9802 - val_loss: 0.0622
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 49s 26ms/step - accuracy: 0.9847 - loss: 0.0500 - val_accuracy: 0.9862 - val_loss: 0.0460
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 51s 27ms/step - accuracy: 0.9885 - loss: 0.0381 - val_accuracy: 0.9856 - val_loss: 0.0499
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 50s 27ms/step - accuracy: 0.9896 - loss: 0.0341 - val_accuracy: 0.9846 - val_loss: 0.0562

Training Completed Successfully.
```

Evaluate the Model:

```
print("STEP 9 : MODEL EVALUATION")
test_loss, test_acc = model.evaluate(test_images,test_labels)
print("\nTest Loss :", test_loss)
print("Test Accuracy :", round(test_acc*100,2),"%")
```

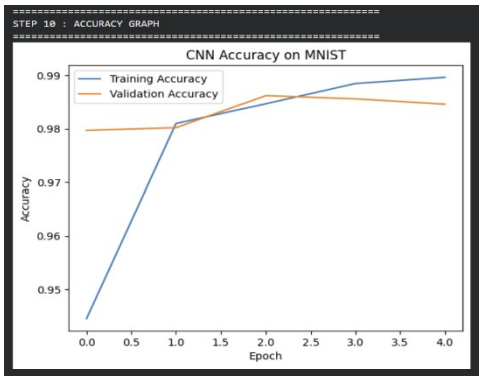
```
============================================================
STEP 9 : MODEL EVALUATION
============================================================
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9846 - loss: 0.0562

Test Loss : 0.056191399693489075
Test Accuracy : 98.46 %
```

Plot Accuracy Graph:

```
print("STEP 10 : ACCURACY GRAPH")
plt.plot(history.history['accuracy'],label='Training Accuracy')
plt.plot(history.history['val_accuracy'],label='Validation Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy on MNIST")
plt.legend()
```

```
plt.show()
```



Predict a Test Image:

```
print("STEP 11 : PREDICTION")
predictions = model.predict(test_images)
index = 0
predicted_digit = np.argmax(predictions[index])
print("Predicted Digit :", predicted_digit)
print("Actual Digit    :", test_labels[index])
print("\nProbability for each digit\n")
for i,p in enumerate(predictions[index]):
print(f"Digit {i} : {p:.4f}")
```

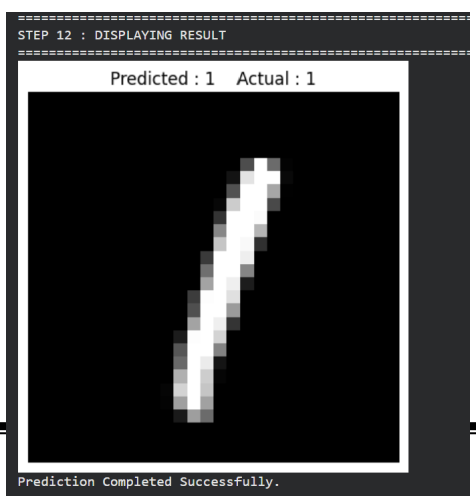
```
============================================================
STEP 11 : PREDICTION
============================================================
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step
Predicted Digit : 1
Actual Digit    : 1

Probability for each digit

Digit 0 : 0.0000
Digit 1 : 1.0000
Digit 2 : 0.0000
Digit 3 : 0.0000
Digit 4 : 0.0000
Digit 5 : 0.0000
Digit 6 : 0.0000
Digit 7 : 0.0000
Digit 8 : 0.0000
Digit 9 : 0.0000
```

Display Predicted Image:

```
print(“STEP 12 : DISPLAYING RESULT”)
plt.imshow(test_images[index].reshape(28,28),cmap=’gray’)
plt.title(
   f”Predicted : {predicted_digit}    Actual : {test_labels[index]}”
)
plt.axis(“off”)
plt.show()
print(“Prediction Completed Successfully.”)
```



EX.NO.2a	CNN VERIFICATION
DATE: 16.07.26	

RESULT:

The Convolutional Neural Network (CNN) was successfully implemented using the MNIST handwritten digit dataset.

AIM:

To verify the performance of the trained Convolutional Neural Network (CNN) model by predicting handwritten digit images from the MNIST test dataset and comparing the predicted labels with the actual labels.

ALGORITHM:

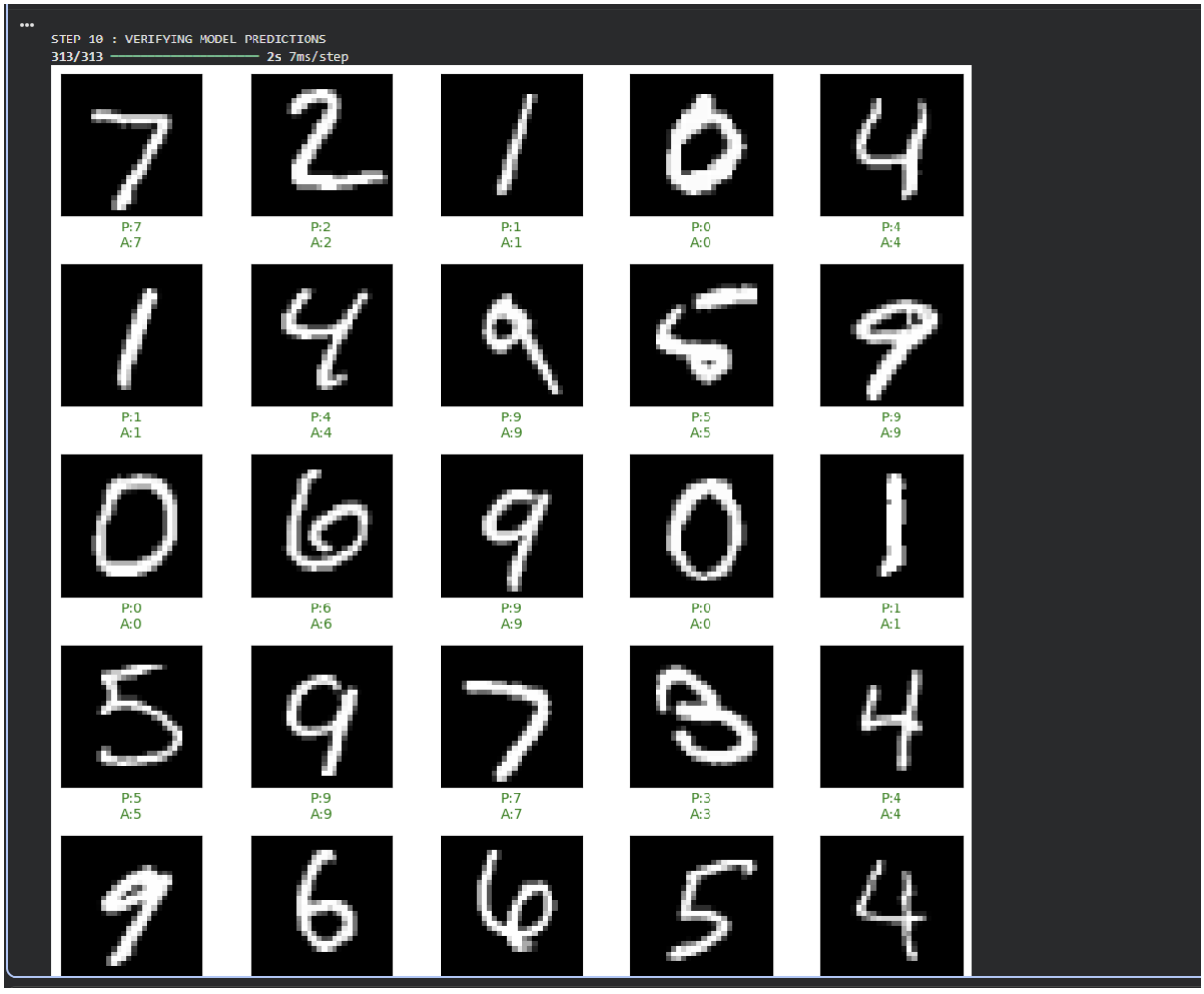
1. Display the title indicating the verification step.
2. Create a figure of size 10 × 10 to display multiple images.
3. Use the trained CNN model to predict the class probabilities of the test images.
4. Loop through the first 25 test images.
5. Divide the figure into a 5 × 5 grid using subplots.
6. Remove the x-axis ticks, y-axis ticks, and grid for better visualization.
7. Display each test image in grayscale.
8. Determine the predicted class using argmax() on the prediction probabilities.
9. Retrieve the actual label from the test dataset.
10. Compare the predicted label with the actual label.
11. Display the predicted label (P) and actual label (A) below each image.
12. Show the label in green if the prediction is correct; otherwise, show it in red.
13. Display all 25 images and print a confirmation message indicating that prediction verification is complete.

PROGRAM:

```
print("\nSTEP 10 : VERIFYING MODEL PREDICTIONS")
plt.figure(figsize=(10,10))
predictions = model.predict(test_images)
for i in range(25):
   plt.subplot(5,5,i+1)
   plt.xticks([])
   plt.yticks([])
   plt.grid(False)
   plt.imshow(test_images[i].reshape(28,28), cmap='gray')
   predicted = np.argmax(predictions[i])
```

```
        actual = test_labels[i]
        color = "green" if predicted == actual else "red"
        plt.xlabel(f"P:{predicted}\nA:{actual}", color=color)
    plt.tight_layout()
    plt.show()
    print("Prediction Verification Completed.")
```

OUTPUT:



EX.NO.2b	CNN VALIDATION
DATE: 16.07.26	

RESULT:

The trained CNN model successfully predicted the handwritten digits from the MNIST test dataset.

AIM:

To validate the performance of the trained Convolutional Neural Network (CNN) model using the MNIST test dataset by calculating the test accuracy, generating a classification report, and displaying the confusion matrix.

ALGORITHM:

1. Import the required libraries: NumPy, Matplotlib, and evaluation metrics from Scikit-learn.
2. Display the title indicating the model validation step.
3. Use the trained CNN model to predict the class probabilities for the test images.
4. Convert the predicted probabilities into class labels using the argmax() function.
5. Compare the predicted labels with the actual test labels to calculate the overall test accuracy.
6. Display the test accuracy as a percentage.
7. Generate and display the Classification Report, which includes:
   - Precision
   - Recall
   - F1-score
   - Support for each digit class (0–9)
8. Generate the Confusion Matrix by comparing the actual and predicted labels.
9. Display the confusion matrix in both numerical and graphical forms.
10. Print a confirmation message indicating that model validation has been completed successfully.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
print("\nSTEP 8 : MODEL VALIDATION")
y_pred_prob = model.predict(test_images)
y_pred = np.argmax(y_pred_prob, axis=1)
accuracy = np.mean(y_pred == test_labels)
print(f"\nTest Accuracy : {accuracy*100:.2f}%")
print("\nClassification Report:")
print(classification_report(test_labels, y_pred))
```

```
cm = confusion_matrix(test_labels, y_pred)
print("\nConfusion Matrix:")
print(cm)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
print("\nModel Validation Completed Successfully.")
```

OUTPUT:

```
STEP 8 : MODEL VALIDATION
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step

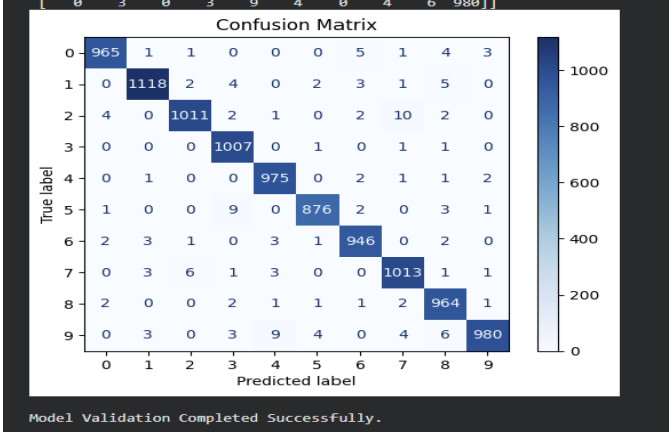
Test Accuracy : 98.55%

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.98      0.99       980
           1       0.99      0.99      0.99      1135
           2       0.99      0.98      0.98      1032
           3       0.98      1.00      0.99      1010
           4       0.98      0.99      0.99       982
           5       0.99      0.98      0.99       892
           6       0.98      0.99      0.99       958
           7       0.98      0.99      0.98      1028
           8       0.97      0.99      0.98       974
           9       0.99      0.97      0.98      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

```
Confusion Matrix:
[[ 965    1    1    0    0    0    5    1    4    3]
 [   0 1118    2    4    0    2    3    1    5    0]
 [   4    0 1011    2    1    0    2   10    2    0]
 [   0    0    0 1007    0    1    0    1    1    0]
 [   0    1    0    0  975    0    2    1    1    2]
 [   1    0    0    9    0  876    2    0    3    1]
 [   2    3    1    0    3    1  946    0    2    0]
 [   0    3    6    1    3    0    0 1013    1    1]
 [   2    0    0    2    1    1    1    2  964    1]
 [   0    3    0    3    9    4    0    4    6  980]]
```




```
Model Validation Completed Successfully.
```

EX.NO.3	CNN DETECTION
DATE: 03.08.26	

RESULT:

The trained CNN model was successfully validated using the MNIST test dataset.

AIM:

To develop and train a Convolutional Neural Network (CNN) for detecting and classifying synthetic images containing Heart, Star, and Diamond shapes, and to predict their bounding boxes.

ALGORITHM:

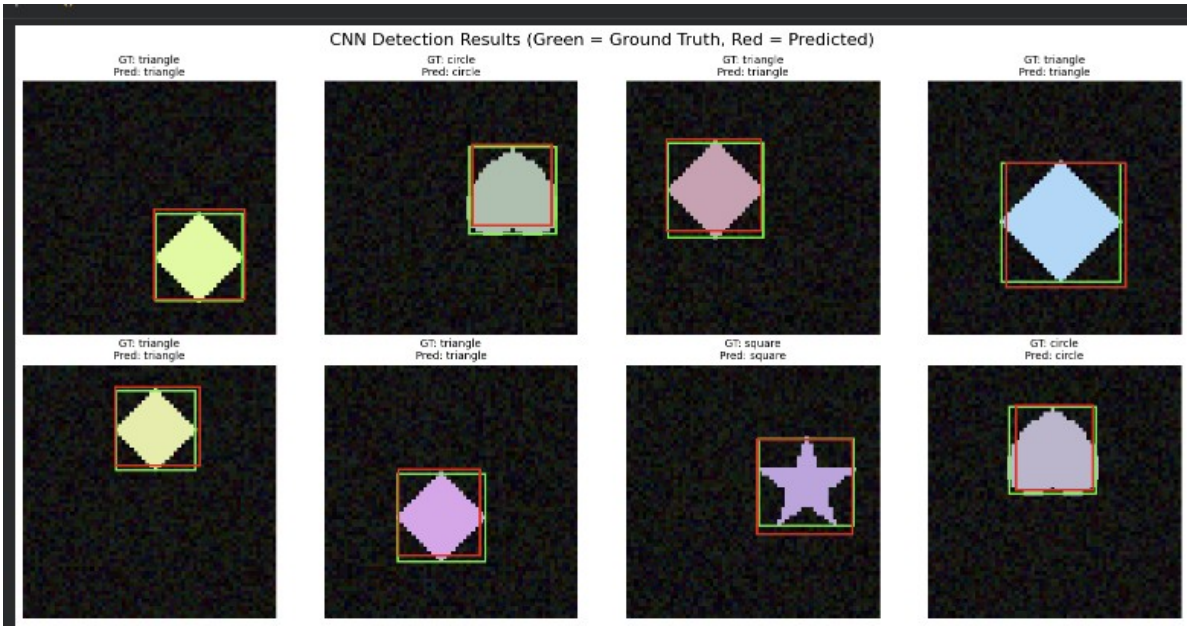
1. Import the required Python libraries.
2. Define the image size and shape classes (Heart, Star, Diamond).
3. Generate a synthetic dataset by randomly drawing one shape in each image.
4. Store the corresponding class label and normalized bounding box coordinates.
5. Build a CNN with shared convolutional layers.
6. Use one output layer for shape classification and another for bounding-box prediction.
7. Train the model using the generated dataset.
8. Evaluate the model using classification accuracy and bounding-box IoU.
9. Display the predicted class and bounding boxes on test images.

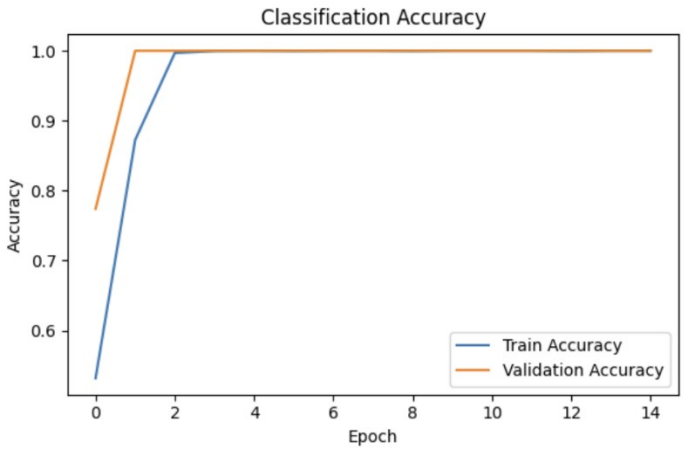
CODE:

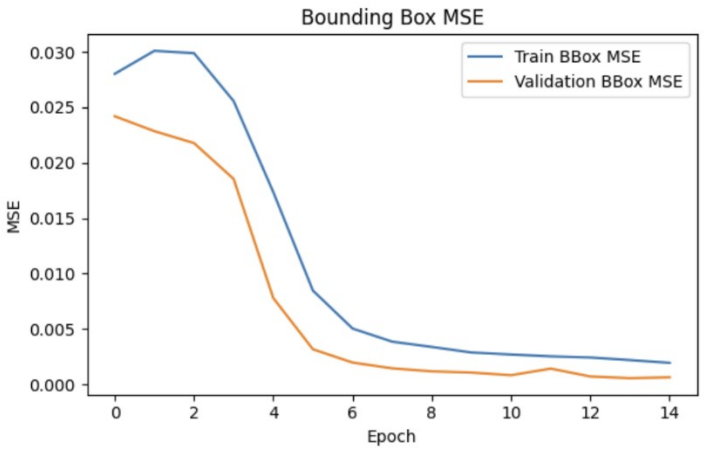
```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
IMG_SIZE = 64
CLASSES = ["Heart", "Star", "Diamond"]
NUM_CLASSES = len(CLASSES)
inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = layers.Conv2D(16, (3,3), activation='relu', padding='same')(inputs)
x = layers.MaxPooling2D((2,2))(x)
x = layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = layers.MaxPooling2D((2,2))(x)
x = layers.Conv2D(64, (3,3), activation='relu', padding='same')(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
class_output = layers.Dense(NUM_CLASSES,
                            activation='softmax',
                            name='class_output')(x)
```

```
bbox_output = layers.Dense(4,
                           activation='sigmoid',
                           name='bbox_output')(x)
model = models.Model(inputs=inputs,
                     outputs=[class_output, bbox_output])
model.compile(
   optimizer='adam',
   loss={
      'class_output':'sparse_categorical_crossentropy',
      'bbox_output':'mse'
   },
   metrics={
      'class_output':'accuracy',
      'bbox_output':'mse'
   }
)
print("CNN Object Detection Model Created Successfully")
model.summary()
```

OUTPUT:







EX.NO.4	CNN SEGMENTATION
DATE: 10.09.26	

RESULT:

The CNN model successfully detected and classified the synthetic Heart, Star, and Diamond shapes.

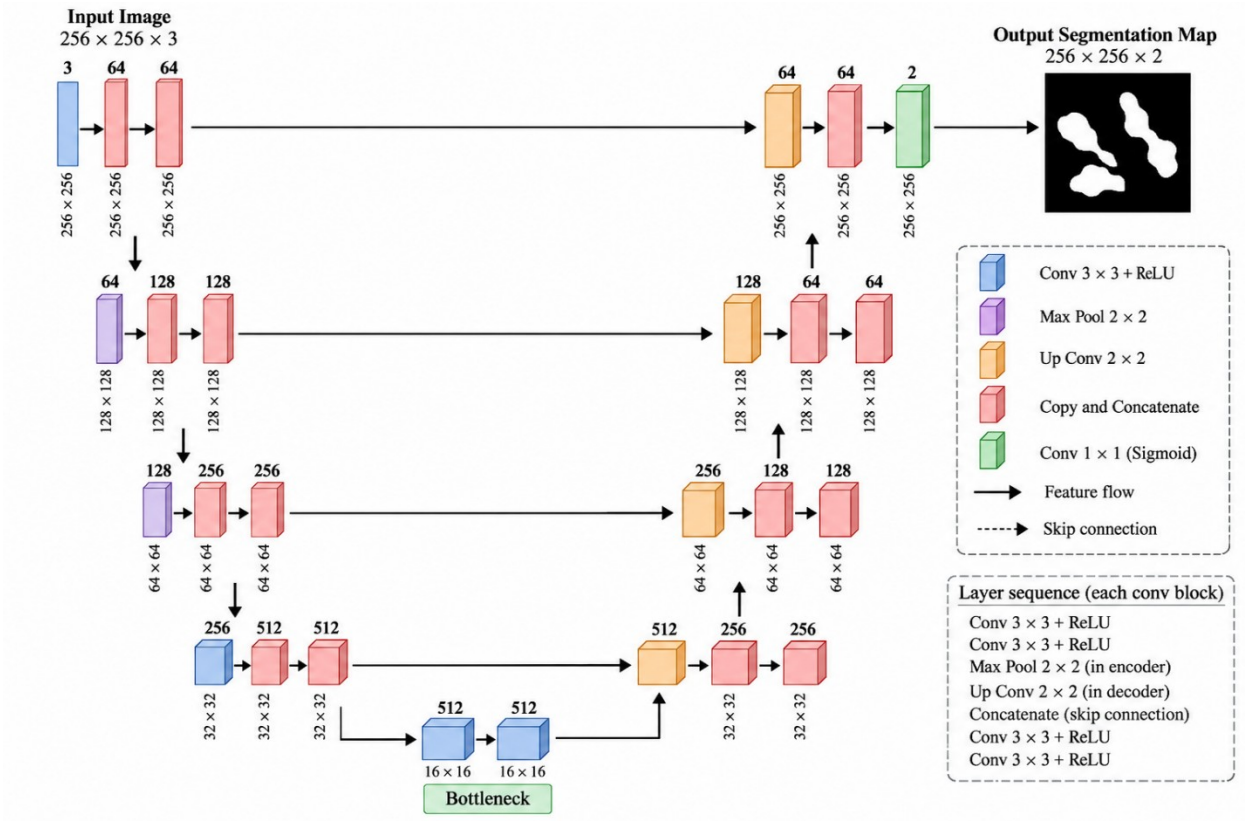
AIM:

To implement an image segmentation model using a Convolutional Neural Network (CNN) and generate a pixel-level segmentation mask for the objects present in an image.

ALGORITHM:

1. Start the program and import the required libraries such as NumPy, Matplotlib, and TensorFlow/Keras.
2. Define the image parameters, such as image size and number of samples.
3. Generate the dataset containing images and their corresponding segmentation masks.
4. Preprocess the images and masks into a suitable format for CNN input.
5. Split the dataset into training and testing sets.
6. Build the CNN model using convolution, pooling, upsampling, and output layers.
7. Compile the model using the Adam optimizer, binary cross-entropy loss, and accuracy metric.
8. Train the CNN model using the training images and their corresponding masks.
9. Evaluate the model using the test dataset.
10. Predict segmentation masks for the test images.
11. Display the original image, actual mask, and predicted mask for comparison.
12. Stop the program.

U – NET ARCHITECTURE :

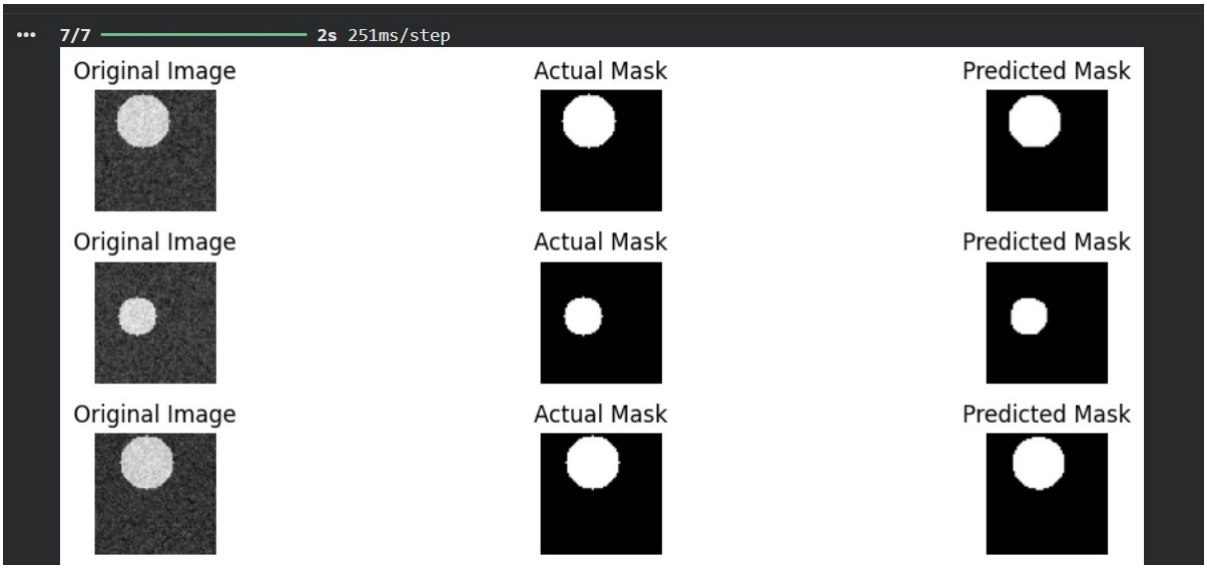


PROGRAM:

```
import os
import numpy as np
import matplotlib.pyplot as plt
import unet
from unet import UNet
image_size = 256
image_channels = 3
image_dir = "dataset"
epochs = 20
batch_size = 8
validation_data_size = 50
test_data_size = 200
train_data = 600
def visualize(image, mask, prediction_img=None):
    fig = plt.figure()
    fig.suptitle("Visualization", fontsize=15)
    ax = fig.add_subplot(1, 3, 1)
    ax.imshow(image)
    ax.set_title("Image")
    ax.axis("off")
    ax = fig.add_subplot(1, 3, 2)
    ax.imshow(
        np.squeeze(mask),
        cmap="gray"
    )
    ax.set_title("Original Mask")
    ax.axis("off")
```

```
        if prediction_img is not None:
            ax = fig.add_subplot(1, 3, 3)
            ax.imshow(
                np.squeeze(prediction_img),
                cmap="gray"
            )
            ax.set_title("Predicted Mask")
            ax.axis("off")
        plt.show()
    ax.set_xticks([])
    ax.set_yticks([])
    temp_data = NucleiDataGenerator(
        img_ids=train_data,
        img_size=image_size,
        batch_size=batch_size
    )
    image, mask = temp_data.__getitem__(0)
    print("Batch shape:", image.shape, mask.shape)
    visualize(
        image=image[2],
        mask=mask[2]
    )
```

OUTPUT:



RESULT:

The CNN-based image segmentation model was successfully implemented.

EX.NO.5	CNN Feature Extraction from Video Frames
DATE: 17.09.26	

AIM:

To extract spatial features from the individual frames of a video using Convolutional Neural Network (CNN) operations such as convolution, ReLU, max pooling and flattening, and to convert every video of the UCF101 dataset into a sequence of 20 CNN feature vectors (20 × 2048) using a pre-trained InceptionV3 network.

DESCRIPTION:

A video is an ordered sequence of frames. Every frame carries spatial information (objects, edges, textures, body posture), while the order of the frames carries temporal information. To classify a video, the spatial content of each frame is first converted into a compact numerical description by a Convolutional Neural Network (CNN). The resulting sequence of feature vectors is then passed to a recurrent network, which is done in Experiments 6 and 7.

Basic CNN operations:

- **Convolution –** small filters (3 × 3 kernels) slide over the image and respond to edges, corners and textures, producing feature maps.
- **ReLU –** f(x) = max(0, x); keeps only the positive responses and adds non-linearity.
- **Max pooling –** a 2 × 2 window keeps the strongest response, halving the height and width of every feature map.
- **Flatten –** converts the final feature maps into a one-dimensional vector that a dense layer or an RNN can read.

Transfer learning with InceptionV3:

Training a deep CNN from scratch needs a very large dataset. Instead, InceptionV3, which is already trained on the ImageNet-1k dataset, is used as a fixed feature extractor. Its classification head is removed (include_top=False) and global average pooling is applied, so every 224 × 224 × 3 frame is described by a 2048-dimensional vector.

Dataset and frame preparation:

A sub-sampled UCF101 dataset with five action classes (CricketShot, PlayingCello, Punch, ShavingBeard and TennisSwing) is used: 594 training videos and 224 test videos. The frames of a video are captured with OpenCV, centre-cropped to a square and resized to 224 × 224. At most MAX_SEQ_LENGTH = 20 frames are kept per video; shorter videos are padded with zeros and a boolean mask records which time-steps are real frames.

ALGORITHM:

1. Download the UCF101 (5-class) subset and read the train.csv and test.csv video lists.
2. Define the hyperparameters IMG_SIZE = 224, MAX_SEQ_LENGTH = 20 and NUM_FEATURES = 2048.
3. Read the frames of a video with OpenCV, centre-crop each frame to a square, resize it to 224 × 224 and convert BGR to RGB.
4. Apply convolution, ReLU, max pooling and flatten to one frame to visualise the basic CNN operations.
5. Load InceptionV3 (ImageNet weights, include_top = False, pooling = "avg") and add its pre-

processing layer to build the feature extractor.

6. Encode the class labels as integers with the StringLookup layer.
7. For every video, pass up to 20 frames through the feature extractor, zero-pad shorter videos and create the frame mask.
8. Save the feature tensors, masks and labels to ucf101_features.npz for the recurrent models.
9. Visualise the feature matrix of a video and display the result.

ARCHITECTURE DIAGRAM:

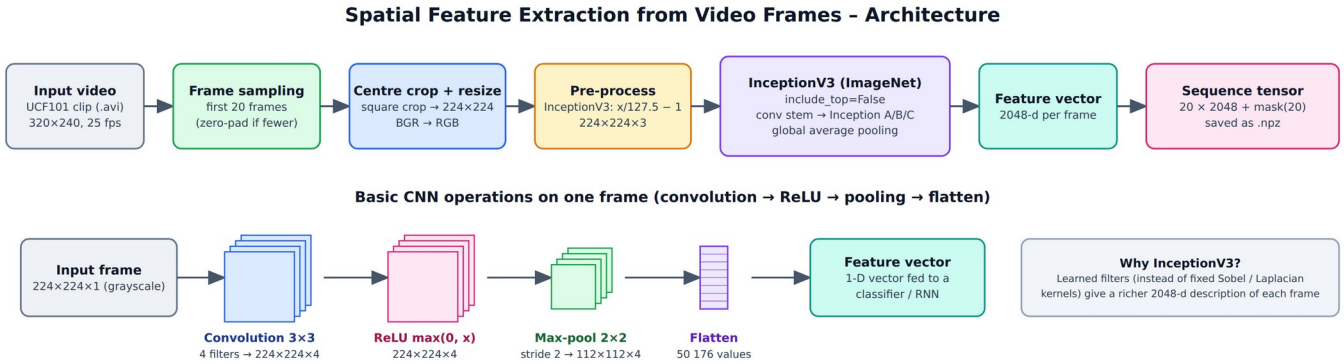


Fig. 5.1 Spatial feature extraction pipeline and basic CNN operations

PROGRAM:

Step 1 : Download the dataset

```
!wget -q https://github.com/sayakpaul/Action-Recognition-in-TensorFlow/releases/download/v1.0.0/
ucf101_top5.tar.gz
!tar xf ucf101_top5.tar.gz
```

Step 2 : Import libraries and define hyperparameters

```
import os
import cv2
import keras
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt

IMG_SIZE = 224
BATCH_SIZE = 64
EPOCHS = 10

MAX_SEQ_LENGTH = 20
NUM_FEATURES = 2048

print("TensorFlow version :", tf.__version__)
print("Keras version      :", keras.__version__)
print("OpenCV version     :", cv2.__version__)
print("IMG_SIZE =", IMG_SIZE, "| MAX_SEQ_LENGTH =", MAX_SEQ_LENGTH, "| NUM_FEATURES =",
NUM_FEATURES)
```

OUTPUT:

```
TensorFlow version : 2.16.2
Keras version      : 3.15.1
OpenCV version     : 4.11.0
IMG_SIZE = 224 | MAX_SEQ_LENGTH = 20 | NUM_FEATURES = 2048
```

Step 3 : Load the train / test video lists

```
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")

print(f"Total videos for training: {len(train_df)}")
print(f"Total videos for testing: {len(test_df)}")

train_df.sample(10)
```

OUTPUT:

```
Total videos for training: 594
Total videos for testing: 224
```

	video_name	tag
445	v_ShavingBeard_g21_c03.avi	ShavingBeard
579	v_TennisSwing_g23_c04.avi	TennisSwing
268	v_Punch_g12_c04.avi	Punch
111	v_CricketShot_g25_c01.avi	CricketShot
22	v_CricketShot_g11_c02.avi	CricketShot
464	v_ShavingBeard_g24_c01.avi	ShavingBeard
436	v_ShavingBeard_g20_c01.avi	ShavingBeard
90	v_CricketShot_g21_c05.avi	CricketShot
415	v_ShavingBeard_g17_c01.avi	ShavingBeard
450	v_ShavingBeard_g22_c01.avi	ShavingBeard

Step 4 : Frame-extraction utilities

```
def crop_center_square(frame):
    y, x = frame.shape[0:2]
    min_dim = min(y, x)
    start_x = (x // 2) - (min_dim // 2)
    start_y = (y // 2) - (min_dim // 2)
    return frame[start_y : start_y + min_dim, start_x : start_x + min_dim]


def load_video(path, max_frames=0, resize=(IMG_SIZE, IMG_SIZE)):
    cap = cv2.VideoCapture(path)
    frames = []
    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break
            frame = crop_center_square(frame)
            frame = cv2.resize(frame, resize)
            frame = frame[:, :, [2, 1, 0]]      # BGR -> RGB
            frames.append(frame)
            if len(frames) == max_frames:
                break
    finally:
        cap.release()
    return np.array(frames)
```

Step 5 : Read frames from one video

```
sample_video = os.path.join("train", train_df["video_name"].iloc[0])
frames = load_video(sample_video)
print("Video path    :", sample_video)
print("Total frames  :", len(frames))
print("Frame shape   :", frames[0].shape)

idx = np.linspace(0, len(frames) - 1, 5, dtype=int)
fig, axes = plt.subplots(1, 5, figsize=(12, 3.1))
for ax, i in zip(axes, idx):
    ax.imshow(frames[i])
    ax.set_title(f"Frame {i}")
    ax.axis("off")
plt.suptitle(f"Frames extracted from {os.path.basename(sample_video)}", fontsize=13)
plt.show()
```

OUTPUT:

```
Video path    : train/v_CricketShot_g08_c01.avi
Total frames  : 75
Frame shape   : (224, 224, 3)
```



Step 6 : Basic CNN operations: convolution, ReLU, max pooling, flatten

```
frame = frames[idx[2]].astype("float32") / 255.0
gray = frame.mean(axis=-1)[None, ..., None]                    # (1, 224, 224, 1)

kernels = {
    "Sobel-X (vertical edges)":   [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]],
    "Sobel-Y (horizontal edges)": [[-1, -2, -1], [0, 0, 0], [1, 2, 1]],
    "Laplacian (blobs)":          [[0, 1, 0], [1, -4, 1], [0, 1, 0]],
    "Gaussian blur":              [[1/16, 2/16, 1/16], [2/16, 4/16, 2/16], [1/16, 2/16, 1/16]],
}
W = np.stack([np.array(k, dtype="float32") for k in kernels.values()], axis=-1)[:, :, None, :]  #
(3,3,1,4)

conv = tf.nn.conv2d(gray, W, strides=1, padding="SAME")           # convolution
relu = tf.nn.relu(conv)                                           # ReLU activation
pool = tf.nn.max_pool2d(relu, ksize=2, strides=2, padding="VALID")# 2x2 max pooling
flat = tf.reshape(pool, (1, -1))                                  # flatten

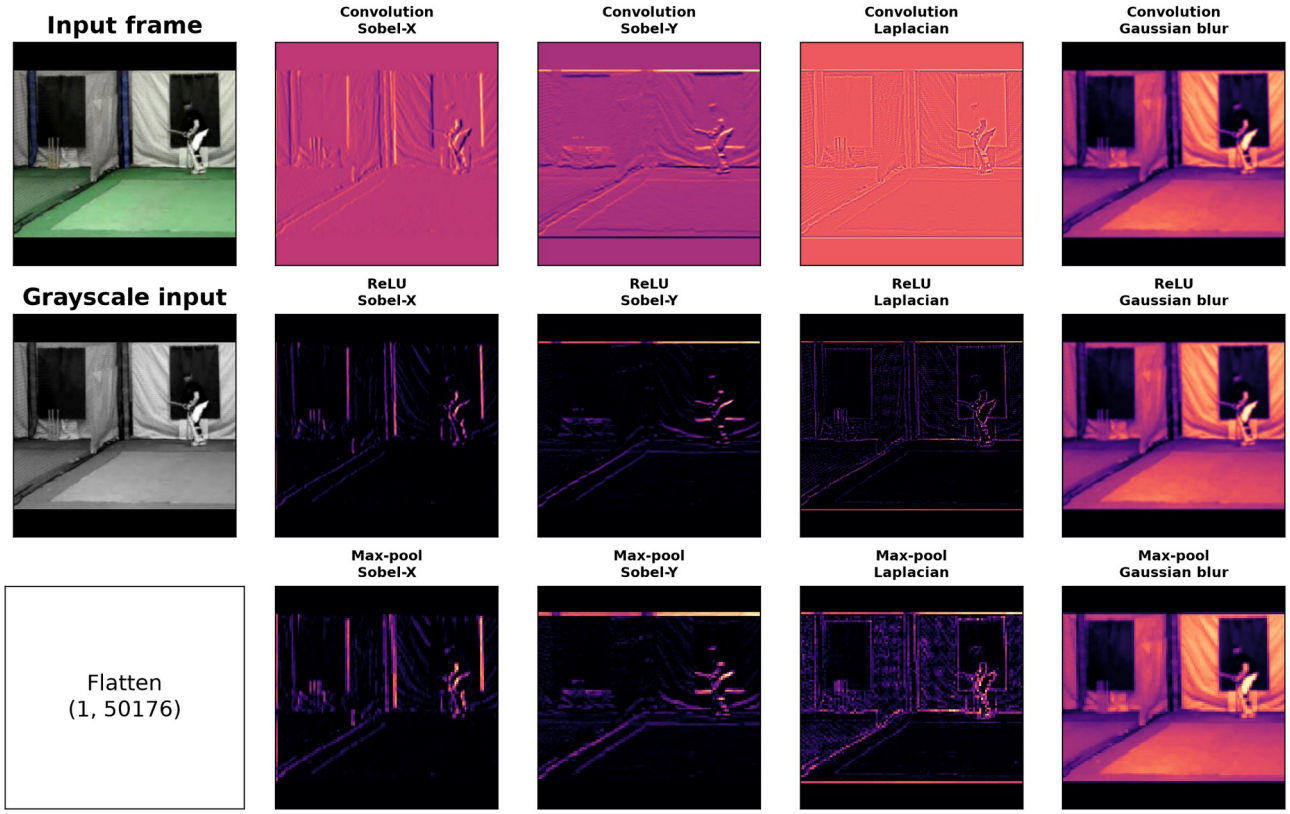
print("Input  (grayscale frame) :", gray.shape)
print("Convolution (4 filters)  :", conv.shape)
print("ReLU                     :", relu.shape)
print("Max pooling (2x2)        :", pool.shape)
print("Flatten                  :", flat.shape)

fig, ax = plt.subplots(3, 5, figsize=(12, 7.6))
ax[0, 0].imshow(frame); ax[0, 0].set_title("Input frame")
for r, (T, name) in enumerate([(conv, "Convolution"), (relu, "ReLU"), (pool, "Max-pool")]):
    for c, k in enumerate(kernels):
        ax[r, c + 1].imshow(T[0, :, :, c], cmap="magma")
        ax[r, c + 1].set_title(f"{name}\n{k.split(' (')[0]}", fontsize=9)
ax[1, 0].imshow(gray[0, :, :, 0], cmap="gray"); ax[1, 0].set_title("Grayscale input")
ax[2, 0].text(0.5, 0.5, f"Flatten\n{tuple(flat.shape)}", ha="center", va="center", fontsize=14);
```

```
ax[2, 0].set_xlim(0, 1); ax[2, 0].set_ylim(0, 1)
for a in ax.ravel(): a.set_xticks([]); a.set_yticks([])
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Input  (grayscale frame) : (1, 224, 224, 1)
Convolution (4 filters)  : (1, 224, 224, 4)
ReLU                     : (1, 224, 224, 4)
Max pooling (2x2)        : (1, 112, 112, 4)
Flatten                  : (1, 50176)
```



Step 7 : Pre-trained CNN (InceptionV3) as feature extractor

```
def build_feature_extractor():
    feature_extractor = keras.applications.InceptionV3(
        weights="imagenet",
        include_top=False,
        pooling="avg",
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
    )
    preprocess_input = keras.applications.inception_v3.preprocess_input

    inputs = keras.Input((IMG_SIZE, IMG_SIZE, 3))
    preprocessed = preprocess_input(inputs)

    outputs = feature_extractor(preprocessed)
    return keras.Model(inputs, outputs, name="feature_extractor")


feature_extractor = build_feature_extractor()
feature_extractor.summary()
```

OUTPUT:

Model: "feature_extractor"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
inception_v3 (Functional)	(None, 2048)	21,802,784

Total params: 21,802,784 (83.17 MB)
Trainable params: 21,768,352 (83.04 MB)
Non-trainable params: 34,432 (134.50 KB)

Step 8 : Encode the class labels

```
label_processor = keras.layers.StringLookup(
    num_oov_indices=0, vocabulary=np.unique(train_df["tag"])
)
print(label_processor.get_vocabulary())
print()
print(train_df["tag"].value_counts().to_string())
```

OUTPUT:

```
['CricketShot', 'PlayingCello', 'Punch', 'ShavingBeard', 'TennisSwing']

tag
Punch           121
PlayingCello    120
CricketShot     118
ShavingBeard    118
TennisSwing     117
```

Step 9 : Video-to-feature utility

```
def prepare_all_videos(df, root_dir):
    num_samples = len(df)
    video_paths = df["video_name"].values.tolist()
    labels = df["tag"].values
    labels = keras.ops.convert_to_numpy(label_processor(labels[..., None]))

    # frame_masks marks which of the MAX_SEQ_LENGTH time-steps hold real frames (1) or padding
(0)
    frame_masks = np.zeros(shape=(num_samples, MAX_SEQ_LENGTH), dtype="bool")
    frame_features = np.zeros(shape=(num_samples, MAX_SEQ_LENGTH, NUM_FEATURES), dtype="float32")

    for idx, path in enumerate(video_paths):
        frames = load_video(os.path.join(root_dir, path), max_frames=MAX_SEQ_LENGTH)
        length = min(MAX_SEQ_LENGTH, frames.shape[0])
        frame_features[idx, :length] = feature_extractor.predict(frames[:length], verbose=0)
        frame_masks[idx, :length] = 1

    return (frame_features, frame_masks), labels
```

Step 10 : Extract CNN features for every video

```
train_data, train_labels = prepare_all_videos(train_df, "train")
test_data, test_labels = prepare_all_videos(test_df, "test")
```

```
print(f"Frame features in train set: {train_data[0].shape}")
print(f"Frame masks in train set: {train_data[1].shape}")
print(f"Frame features in test set : {test_data[0].shape}")
print(f"Frame masks in test set : {test_data[1].shape}")

np.savez_compressed("ucf101_features.npz",
                    train_features=train_data[0], train_masks=train_data[1],
train_labels=train_labels,
                    test_features=test_data[0], test_masks=test_data[1], test_labels=test_labels)
print("Features saved to ucf101_features.npz")
```

OUTPUT:

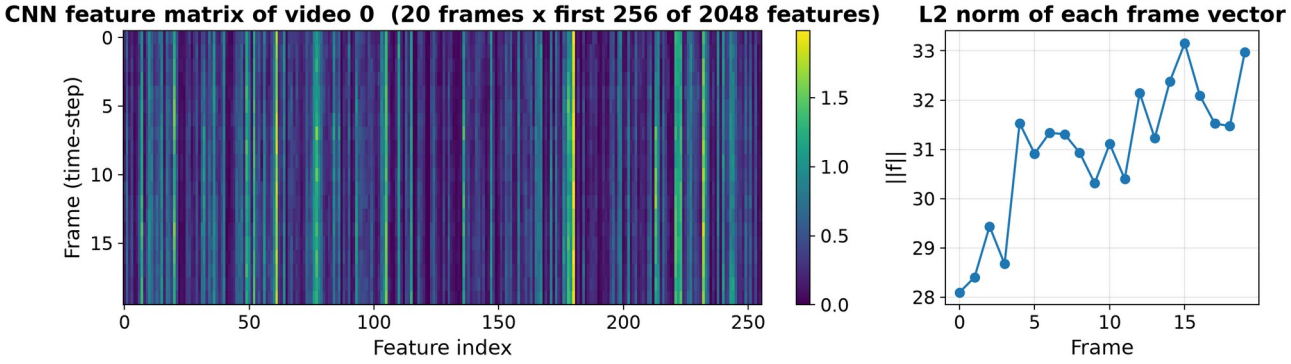
```
Frame features in train set: (594, 20, 2048)
Frame masks in train set: (594, 20)
Frame features in test set : (224, 20, 2048)
Frame masks in test set : (224, 20)
Features saved to ucf101_features.npz
```

Step 11 : Visualise the extracted feature sequence

```
fig, ax = plt.subplots(1, 2, figsize=(12, 3.8), gridspec_kw={"width_ratios": [2.2, 1]})
im = ax[0].imshow(train_data[0][0][:, :256], aspect="auto", cmap="viridis")
ax[0].set_title("CNN feature matrix of video 0  (20 frames x first 256 of 2048 features)")
ax[0].set_xlabel("Feature index"); ax[0].set_ylabel("Frame (time-step)")
plt.colorbar(im, ax=ax[0], fraction=0.025)
ax[1].plot(np.linalg.norm(train_data[0][0], axis=1), marker="o", color="#1f77b4")
ax[1].set_title("L2 norm of each frame vector"); ax[1].set_xlabel("Frame"); ax[1].set_ylabel("||
f||"); ax[1].grid(alpha=.3)
plt.tight_layout()
plt.show()
print("Mean activation :", round(float(train_data[0][train_data[1] == 1].mean()), 4))
print("Feature vector per frame :", NUM_FEATURES, "-> sequence per video :", MAX_SEQ_LENGTH, "x",
NUM_FEATURES)
```

OUTPUT:

```
Mean activation : 0.4915
Feature vector per frame : 2048 -> sequence per video : 20 x 2048
```



RESULT:

Thus, the spatial features of the video frames were extracted successfully using convolution, ReLU, max pooling and flatten operations and the pre-trained InceptionV3 CNN. All 594 training videos and 224 test videos of the UCF101 subset were converted into frame-feature sequences of shape (20, 2048) and saved for the temporal models of the next experiments.

EX.NO.6	RNN/LSTM Temporal Sequence Modeling
DATE: 17.09.26	

AIM:

To model the temporal relationship between consecutive video frames with Recurrent Neural Networks (GRU and LSTM) that read the CNN frame features produced in Experiment 5, and to compare the two recurrent architectures for classifying UCF101 action videos.

DESCRIPTION:

A Recurrent Neural Network (RNN) reads a sequence one element at a time and keeps a hidden state h(t) that summarises everything seen so far. Because a plain RNN suffers from the vanishing-gradient problem on long sequences, gated variants such as the GRU and the LSTM are used.

Gated Recurrent Unit (GRU):

A GRU has an update gate z(t) and a reset gate r(t). They decide how much of the previous state is kept and how much new information is written:

$$z(t) = \sigma(Wz \cdot x(t) + Uz \cdot h(t-1)), \quad r(t) = \sigma(Wr \cdot x(t) + Ur \cdot h(t-1))$$

$$\tilde{h}(t) = \tanh(W \cdot x(t) + U \cdot (r(t) \odot h(t-1))), \quad h(t) = (1 - z(t)) \odot h(t-1) + z(t) \odot \tilde{h}(t)$$

Long Short-Term Memory (LSTM):

An LSTM keeps a separate cell state c(t) and uses three gates – forget, input and output – to control what is erased, written and exposed. It has four weight sets instead of the three of a GRU, so it has more parameters.

Model used:

The input of the model is the (20 × 2048) feature sequence of a video and a mask of 20 booleans that hides the zero-padded time-steps. Two recurrent layers (16 and 8 units) are followed by Dropout (0.4), a Dense layer of 8 ReLU units and a 5-way softmax layer. The model is compiled with sparse categorical cross-entropy and the Adam optimizer. Training runs for 10 epochs with 30 % of the training data used for validation; only the best weights (lowest validation loss) are checkpointed and later used for testing.

ALGORITHM:

1. Load the CNN feature sequences, masks and labels saved in Experiment 5.
2. Shuffle the training videos once, so that the validation split contains videos of every class.
3. Build the sequence model: input → GRU(16, return_sequences) → GRU(8) → Dropout(0.4) → Dense(8, ReLU) → Dense(5, softmax).
4. Compile the model with sparse categorical cross-entropy loss, Adam optimizer and accuracy metric.
5. Train for 10 epochs with validation_split = 0.3 and save the best weights with ModelCheckpoint.
6. Reload the best weights and evaluate the model on the 224 test videos.
7. Plot the accuracy and loss curves.
8. Build the same network with LSTM layers, train it and compare the test accuracy of the GRU and the LSTM.

ARCHITECTURE DIAGRAM:

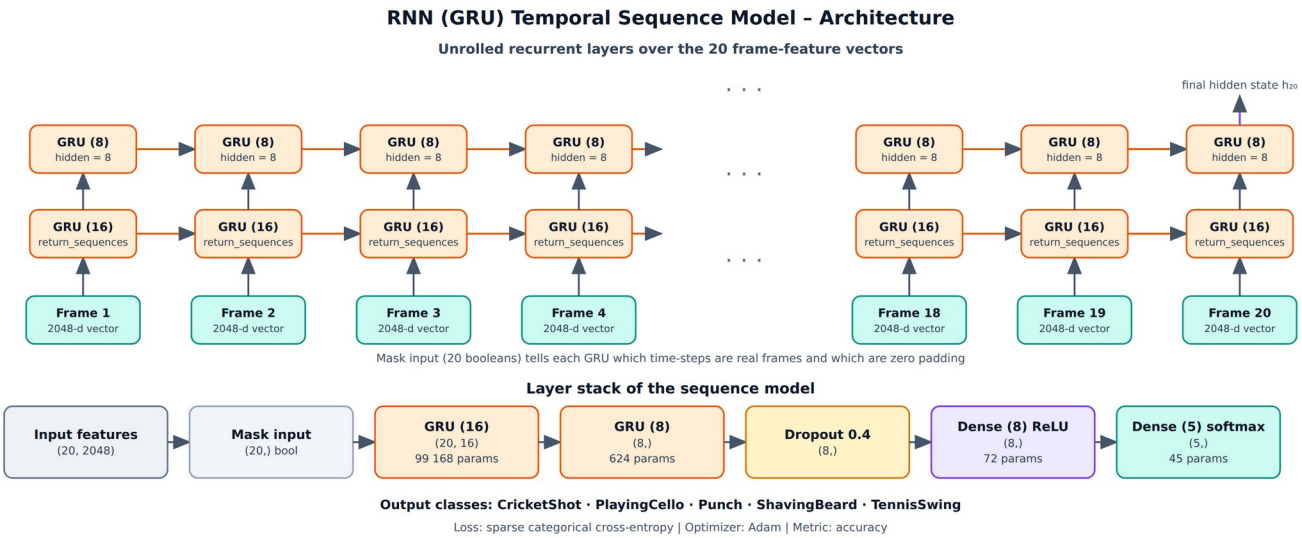


Fig. 6.1 Unrolled GRU layers and layer stack of the temporal model

PROGRAM:

Step 1 : Import libraries and load the CNN feature sequences

```
import os
import keras
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt

IMG_SIZE = 224
BATCH_SIZE = 64
EPOCHS = 10

MAX_SEQ_LENGTH = 20
NUM_FEATURES = 2048

keras.utils.set_random_seed(42)

data = np.load("ucf101_features.npz")
train_data, train_labels = (data["train_features"], data["train_masks"]), data["train_labels"]
test_data, test_labels = (data["test_features"], data["test_masks"]), data["test_labels"]
class_vocab = sorted(pd.read_csv("train.csv")["tag"].unique())

# train.csv is sorted class-by-class, but validation_split takes the LAST 30 % of the data.
# Shuffle once so that the validation set contains videos of every class.
perm = np.random.RandomState(42).permutation(len(train_labels))
train_data = (train_data[0][perm], train_data[1][perm])
train_labels = train_labels[perm]

print("Class vocabulary           :", class_vocab)
print("Frame features (train)     :", train_data[0].shape)
print("Frame masks    (train)     :", train_data[1].shape)
print("Labels         (train)     :", train_labels.shape)
print("Frame features (test)      :", test_data[0].shape)
```

OUTPUT:

```
Class vocabulary           : ['CricketShot', 'PlayingCello', 'Punch', 'ShavingBeard', 'TennisSwing']
Frame features (train)     : (594, 20, 2048)
Frame masks    (train)     : (594, 20)
Labels         (train)     : (594, 1)
Frame features (test)      : (224, 20, 2048)
```

Step 2 : Sequence model with GRU layers

```
def get_sequence_model():
    frame_features_input = keras.Input((MAX_SEQ_LENGTH, NUM_FEATURES))
    mask_input = keras.Input((MAX_SEQ_LENGTH,), dtype="bool")

    # the mask hides the zero-padded time-steps from the recurrent layers
    x = keras.layers.GRU(16, return_sequences=True)(frame_features_input, mask=mask_input)
    x = keras.layers.GRU(8)(x)
    x = keras.layers.Dropout(0.4)(x)
    x = keras.layers.Dense(8, activation="relu")(x)
    output = keras.layers.Dense(len(class_vocab), activation="softmax")(x)

    rnn_model = keras.Model([frame_features_input, mask_input], output)
    rnn_model.compile(loss="sparse_categorical_crossentropy", optimizer="adam",
metrics=["accuracy"])
    return rnn_model

get_sequence_model().summary()
```

OUTPUT:

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 20, 2048)	0	-
input_layer_1 (InputLayer)	(None, 20)	0	-
gru (GRU)	(None, 20, 16)	99,168	input_layer[0][0… input_layer_1[0]…
gru_1 (GRU)	(None, 8)	624	gru[0][0], input_layer_1[0]…
dropout (Dropout)	(None, 8)	0	gru_1[0][0]
dense (Dense)	(None, 8)	72	dropout[0][0]
dense_1 (Dense)	(None, 5)	45	dense[0][0]

Total params: 99,909 (390.27 KB)
Trainable params: 99,909 (390.27 KB)
Non-trainable params: 0 (0.00 B)

Step 3 : Train the RNN (GRU) model

```
def run_experiment():
    os.makedirs("/tmp/video_classifier", exist_ok=True)
    filepath = "/tmp/video_classifier/ckpt.weights.h5"
    checkpoint = keras.callbacks.ModelCheckpoint(filepath, save_weights_only=True,
save_best_only=True, verbose=1)

    seq_model = get_sequence_model()
    history = seq_model.fit(
        [train_data[0], train_data[1]],
        train_labels,
        validation_split=0.3,
        epochs=EPOCHS,
        callbacks=[checkpoint],
    )

    seq_model.load_weights(filepath)
    _, accuracy = seq_model.evaluate([test_data[0], test_data[1]], test_labels)
```

```
    print(f"Test accuracy: {round(accuracy * 100, 2)}%")
    return history, seq_model

history, sequence_model = run_experiment()
```

OUTPUT:

```
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.3542 - loss: 1.4183
Epoch 1: val_loss improved from None to 1.16442, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 1: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 4s 84ms/step - accuracy: 0.3542 - loss: 1.4183 - val_accuracy: 0.5084 - val_loss: 1.1644
Epoch 2/10
11/13 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.5227 - loss: 1.2052
Epoch 2: val_loss improved from 1.16442 to 1.03525, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 2: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.5229 - loss: 1.1954 - val_accuracy: 0.7263 - val_loss: 1.0352
Epoch 3/10
12/13 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.5938 - loss: 1.0917
Epoch 3: val_loss improved from 1.03525 to 0.91882, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 3: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.5976 - loss: 1.0905 - val_accuracy: 0.7877 - val_loss: 0.9188
Epoch 4/10
12/13 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.6641 - loss: 0.9916
Epoch 4: val_loss improved from 0.91882 to 0.82673, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 4: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.6723 - loss: 0.9869 - val_accuracy: 0.9218 - val_loss: 0.8267
Epoch 5/10
11/13 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.7557 - loss: 0.8974
Epoch 5: val_loss improved from 0.82673 to 0.75930, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 5: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.7542 - loss: 0.8900 - val_accuracy: 0.9721 - val_loss: 0.7593
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.7711 - loss: 0.8712
Epoch 6: val_loss improved from 0.75930 to 0.69940, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 6: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.7711 - loss: 0.8712 - val_accuracy: 1.0000 - val_loss: 0.6994
Epoch 7/10
12/13 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.8490 - loss: 0.7697
Epoch 7: val_loss improved from 0.69940 to 0.63538, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 7: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.8482 - loss: 0.7704 - val_accuracy: 1.0000 - val_loss: 0.6354
Epoch 8/10
11/13 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.8722 - loss: 0.7229
Epoch 8: val_loss improved from 0.63538 to 0.58583, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 8: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.8747 - loss: 0.7206 - val_accuracy: 1.0000 - val_loss: 0.5858
Epoch 9/10
12/13 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.9036 - loss: 0.6738
Epoch 9: val_loss improved from 0.58583 to 0.53552, saving model to /tmp/video_classifier/ckpt.weights.h5

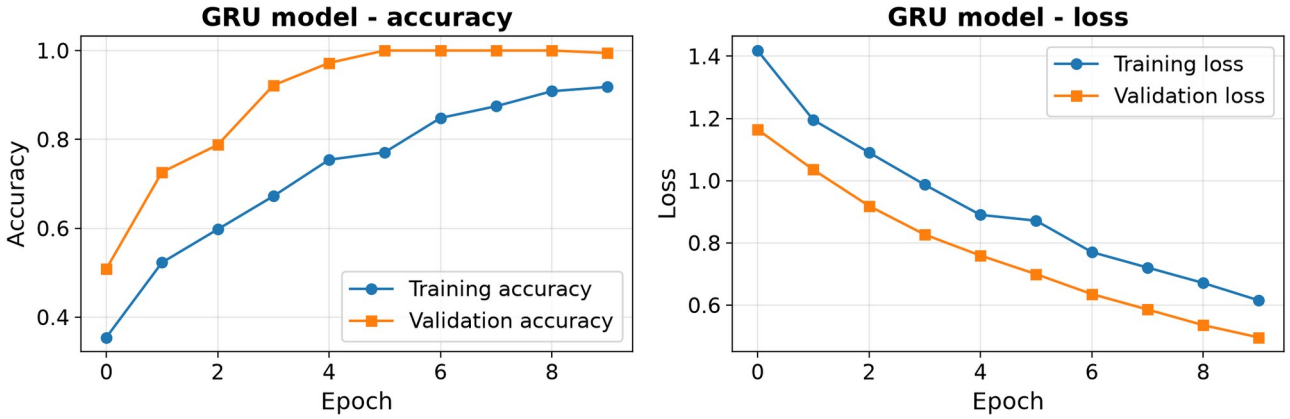
Epoch 9: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9084 - loss: 0.6710 - val_accuracy: 1.0000 - val_loss: 0.5355
Epoch 10/10
11/13 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.9176 - loss: 0.6109
Epoch 10: val_loss improved from 0.53552 to 0.49577, saving model to /tmp/video_classifier/ckpt.weights.h5

Epoch 10: finished saving model to /tmp/video_classifier/ckpt.weights.h5
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - accuracy: 0.9181 - loss: 0.6150 - val_accuracy: 0.9944 - val_loss: 0.4958
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9286 - loss: 0.6121
Test accuracy: 92.86%
```

Step 4 : Accuracy and loss curves

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
ax[0].plot(history.history["accuracy"], marker="o", label="Training accuracy", color="#1f77b4")
ax[0].plot(history.history["val_accuracy"], marker="s", label="Validation accuracy",
color="#ff7f0e")
ax[0].set_title("GRU model - accuracy"); ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Accuracy");
ax[0].legend(); ax[0].grid(alpha=.3)
ax[1].plot(history.history["loss"], marker="o", label="Training loss", color="#1f77b4")
ax[1].plot(history.history["val_loss"], marker="s", label="Validation loss", color="#ff7f0e")
ax[1].set_title("GRU model - loss"); ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Loss");
ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout()
plt.show()
```

OUTPUT:



Step 5 : LSTM variant of the temporal model

```
def get_lstm_model():
    frame_features_input = keras.Input((MAX_SEQ_LENGTH, NUM_FEATURES))
    mask_input = keras.Input((MAX_SEQ_LENGTH,), dtype="bool")
    x = keras.layers.LSTM(16, return_sequences=True)(frame_features_input, mask=mask_input)
    x = keras.layers.LSTM(8)(x)
    x = keras.layers.Dropout(0.4)(x)
    x = keras.layers.Dense(8, activation="relu")(x)
    output = keras.layers.Dense(len(class_vocab), activation="softmax")(x)
    model = keras.Model([frame_features_input, mask_input], output)
    model.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model

lstm_model = get_lstm_model()
lstm_model.summary()
lstm_history = lstm_model.fit([train_data[0], train_data[1]], train_labels,
                              validation_split=0.3, epochs=EPOCHS, verbose=2)
_, lstm_acc = lstm_model.evaluate([test_data[0], test_data[1]], test_labels, verbose=0)
print(f"LSTM test accuracy: {round(lstm_acc * 100, 2)}%")
```

OUTPUT:

Model: "functional_2"

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 20, 2048)	0	-
input_layer_5 (InputLayer)	(None, 20)	0	-
lstm (LSTM)	(None, 20, 16)	132,160	input_layer_4[0]… input_layer_5[0]…
lstm_1 (LSTM)	(None, 8)	800	lstm[0][0], input_layer_5[0]…
dropout_2 (Dropout)	(None, 8)	0	lstm_1[0][0]
dense_4 (Dense)	(None, 8)	72	dropout_2[0][0]
dense_5 (Dense)	(None, 5)	45	dense_4[0][0]

```
 Total params: 133,077 (519.83 KB)
 Trainable params: 133,077 (519.83 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
13/13 - 4s - 324ms/step - accuracy: 0.2699 - loss: 1.5473 - val_accuracy: 0.5531 - val_loss: 1.4416
Epoch 2/10
13/13 - 0s - 37ms/step - accuracy: 0.4530 - loss: 1.4098 - val_accuracy: 0.6369 - val_loss: 1.3187
Epoch 3/10
13/13 - 1s - 40ms/step - accuracy: 0.5422 - loss: 1.3083 - val_accuracy: 0.6927 - val_loss: 1.1906
Epoch 4/10
13/13 - 0s - 36ms/step - accuracy: 0.6361 - loss: 1.1893 - val_accuracy: 0.7765 - val_loss: 1.0979
Epoch 5/10
13/13 - 0s - 36ms/step - accuracy: 0.6361 - loss: 1.1389 - val_accuracy: 0.8492 - val_loss: 0.9974
Epoch 6/10
13/13 - 1s - 46ms/step - accuracy: 0.6916 - loss: 1.0142 - val_accuracy: 0.8492 - val_loss: 0.8898
Epoch 7/10
13/13 - 0s - 34ms/step - accuracy: 0.7446 - loss: 0.9360 - val_accuracy: 0.9497 - val_loss: 0.8063
Epoch 8/10
13/13 - 0s - 38ms/step - accuracy: 0.7373 - loss: 0.8879 - val_accuracy: 0.8994 - val_loss: 0.7681
Epoch 9/10
13/13 - 0s - 37ms/step - accuracy: 0.7928 - loss: 0.8116 - val_accuracy: 0.9721 - val_loss: 0.6796
Epoch 10/10
13/13 - 0s - 35ms/step - accuracy: 0.7904 - loss: 0.8055 - val_accuracy: 0.9385 - val_loss: 0.6091
LSTM test accuracy: 87.95%
```

Step 6 : Compare GRU and LSTM

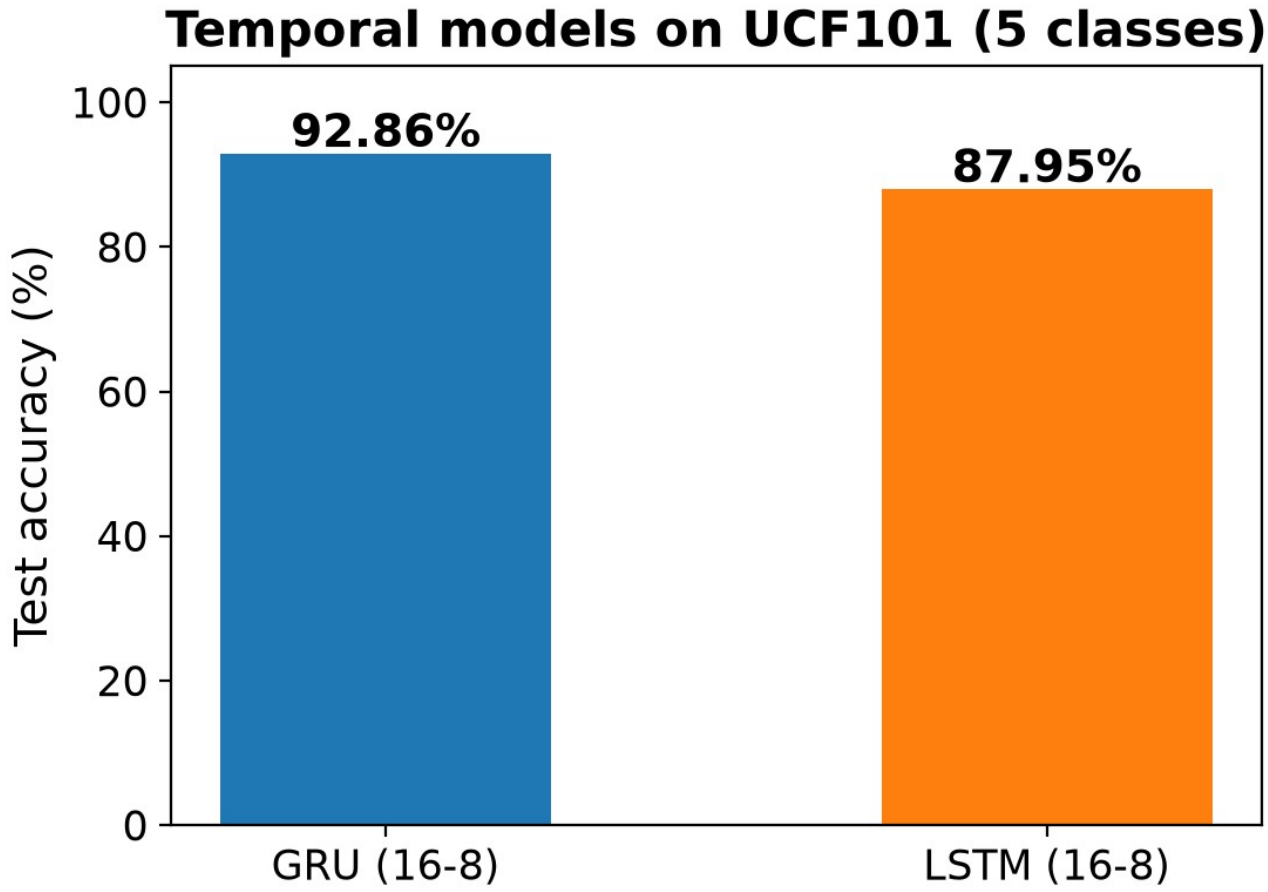
```
_, gru_acc = sequence_model.evaluate([test_data[0], test_data[1]], test_labels, verbose=0)
names = ["GRU (16-8)", "LSTM (16-8)"]
accs = [gru_acc * 100, lstm_acc * 100]
params = [sequence_model.count_params(), lstm_model.count_params()]

print(f"{'Model':<14}{'Parameters':>12}{'Test accuracy':>16}")
for n, p, a in zip(names, params, accs):
    print(f"{n:<14}{p:>12,}{a:>15.2f}%")

plt.figure(figsize=(6, 4.2))
bars = plt.bar(names, accs, color=["#1f77b4", "#ff7f0e"], width=0.5)
for b, a in zip(bars, accs):
    plt.text(b.get_x() + b.get_width() / 2, a + 1, f"{a:.2f}%", ha="center", fontweight="bold")
plt.ylim(0, 105); plt.ylabel("Test accuracy (%)"); plt.title("Temporal models on UCF101 (5
classes)")
plt.show()
```

OUTPUT:

Model	Parameters	Test accuracy
GRU (16-8)	99,909	92.86%
LSTM (16-8)	133,077	87.95%



RESULT:

Thus, the temporal relationship between video frames was modelled successfully with recurrent networks. The GRU model (99,909 parameters) reached a test accuracy of 92.86 % and the LSTM model (133,077 parameters) reached 87.95 % on the 224 unseen test videos, which shows that an RNN on top of CNN frame features recognises the five UCF101 actions reliably.

EX.NO.7	End-to-End CNN+RNN Video Classification Pipeline
DATE: 24.09.26	

AIM:

To implement an end-to-end CNN + RNN video classification pipeline in which a CNN (InceptionV3) extracts the spatial features of the frames, an RNN (GRU) learns the temporal pattern of the frame sequence, and the complete model classifies a raw video into one of the five UCF101 action classes.

DESCRIPTION:

The pipeline of Experiments 5 and 6 is combined into one system that accepts a raw video file and returns the class probabilities. The video is read with OpenCV, 20 frames are captured, the CNN converts each frame into a 2048-dimensional vector (spatial features), and the GRU network reads the 20 vectors in order (temporal features) to produce the final decision.

Both parts are needed: the CNN alone judges every frame separately and ignores motion, while the RNN alone cannot interpret raw pixels. Working together, the CNN describes what is visible in each frame and the RNN describes how it changes over time.

Evaluation measures:

- **Accuracy –** fraction of test videos classified correctly.
- **Precision / Recall / F1-score –** class-wise quality of the predictions, computed with scikit-learn.
- **Confusion matrix –** rows are the true classes and columns the predicted classes; the diagonal shows the correct predictions.

ALGORITHM:

1. Import the libraries and read the video lists; build the vocabulary of the five action classes.
2. Define load_video() and crop_center_square() to read, crop and resize the frames of a video.
3. Build the InceptionV3 feature extractor (spatial part).
4. Build the GRU sequence model (temporal part) and load the best weights saved in Experiment 6.
5. Evaluate the complete model on the 224 test videos.
6. Generate the classification report and the confusion matrix.
7. For a single video: load 20 frames, extract their CNN features, create the mask and predict the class probabilities with the GRU model.
8. Classify one test video of every class and display the frames with the predicted probabilities.
9. Print the class-wise accuracy summary.

ARCHITECTURE DIAGRAM:

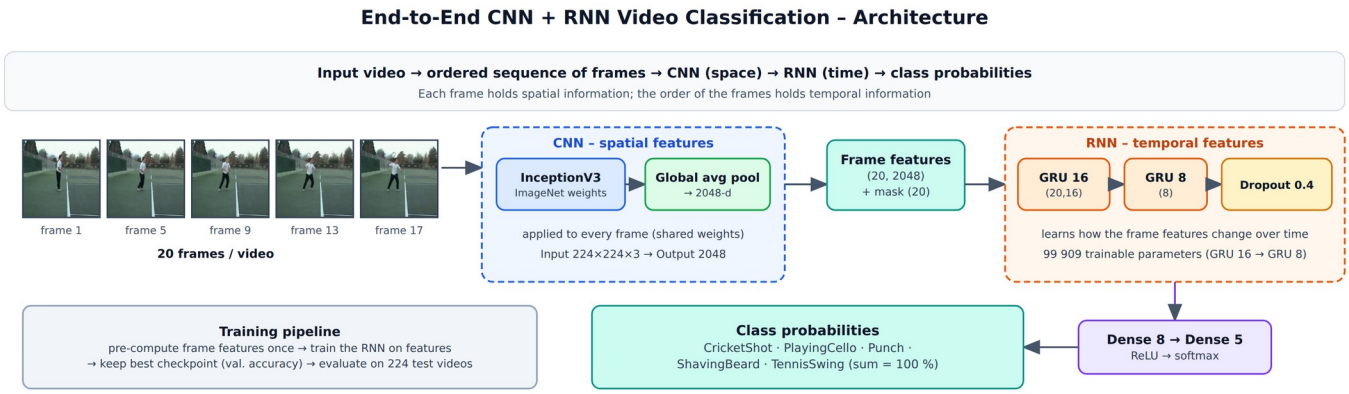


Fig. 7.1 End-to-end CNN + RNN video classification architecture

PROGRAM:

Step 1 : Import libraries and hyperparameters

```
import os
import cv2
import keras
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

IMG_SIZE = 224
MAX_SEQ_LENGTH = 20
NUM_FEATURES = 2048
keras.utils.set_random_seed(42)

train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")
class_vocab = sorted(train_df["tag"].unique())
print("Classes :", class_vocab)
print("Train videos :", len(train_df), "| Test videos :", len(test_df))
```

OUTPUT:

```
Classes : ['CricketShot', 'PlayingCello', 'Punch', 'ShavingBeard', 'TennisSwing']
Train videos : 594 | Test videos : 224
```

Step 2 : Video loading utilities (spatial part)

```
def crop_center_square(frame):
    y, x = frame.shape[0:2]
    min_dim = min(y, x)
    start_x = (x // 2) - (min_dim // 2)
    start_y = (y // 2) - (min_dim // 2)
    return frame[start_y : start_y + min_dim, start_x : start_x + min_dim]


def load_video(path, max_frames=0, resize=(IMG_SIZE, IMG_SIZE)):
    cap = cv2.VideoCapture(path)
    frames = []
    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break
            frame = crop_center_square(frame)
            frame = cv2.resize(frame, resize)
```

```
            frame = frame[:, :, [2, 1, 0]]
            frames.append(frame)
            if len(frames) == max_frames:
                break
    finally:
        cap.release()
    return np.array(frames)


def build_feature_extractor():
    feature_extractor = keras.applications.InceptionV3(
        weights="imagenet", include_top=False, pooling="avg", input_shape=(IMG_SIZE, IMG_SIZE,
3))
    preprocess_input = keras.applications.inception_v3.preprocess_input
    inputs = keras.Input((IMG_SIZE, IMG_SIZE, 3))
    preprocessed = preprocess_input(inputs)
    outputs = feature_extractor(preprocessed)
    return keras.Model(inputs, outputs, name="feature_extractor")


feature_extractor = build_feature_extractor()
print("CNN feature extractor ready  :", feature_extractor.name, "->",
feature_extractor.output_shape)
```

OUTPUT:

```
CNN feature extractor ready  : feature_extractor -> (None, 2048)
```

Step 3 : Sequence model (temporal part) and the best checkpoint

```
def get_sequence_model():
    frame_features_input = keras.Input((MAX_SEQ_LENGTH, NUM_FEATURES))
    mask_input = keras.Input((MAX_SEQ_LENGTH,), dtype="bool")
    x = keras.layers.GRU(16, return_sequences=True)(frame_features_input, mask=mask_input)
    x = keras.layers.GRU(8)(x)
    x = keras.layers.Dropout(0.4)(x)
    x = keras.layers.Dense(8, activation="relu")(x)
    output = keras.layers.Dense(len(class_vocab), activation="softmax")(x)
    rnn_model = keras.Model([frame_features_input, mask_input], output)
    rnn_model.compile(loss="sparse_categorical_crossentropy", optimizer="adam",
metrics=["accuracy"])
    return rnn_model


data = np.load("ucf101_features.npz")
test_data, test_labels = (data["test_features"], data["test_masks"]), data["test_labels"]

sequence_model = get_sequence_model()
sequence_model.load_weights("/tmp/video_classifier/ckpt.weights.h5")      # best weights saved in
Exp. 6
_, accuracy = sequence_model.evaluate([test_data[0], test_data[1]], test_labels)
print(f"Test accuracy: {round(accuracy * 100, 2)}%")
```

OUTPUT:

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - accuracy: 0.9286 - loss: 0.6121
Test accuracy: 92.86%
```

Step 4 : Classification report and confusion matrix

```
probs = sequence_model.predict([test_data[0], test_data[1]], verbose=0)
y_pred = probs.argmax(axis=1)
y_true = test_labels.ravel()

print(classification_report(y_true, y_pred, target_names=class_vocab, digits=4))

cm = confusion_matrix(y_true, y_pred)
fig, ax = plt.subplots(figsize=(6.6, 5.6))
```

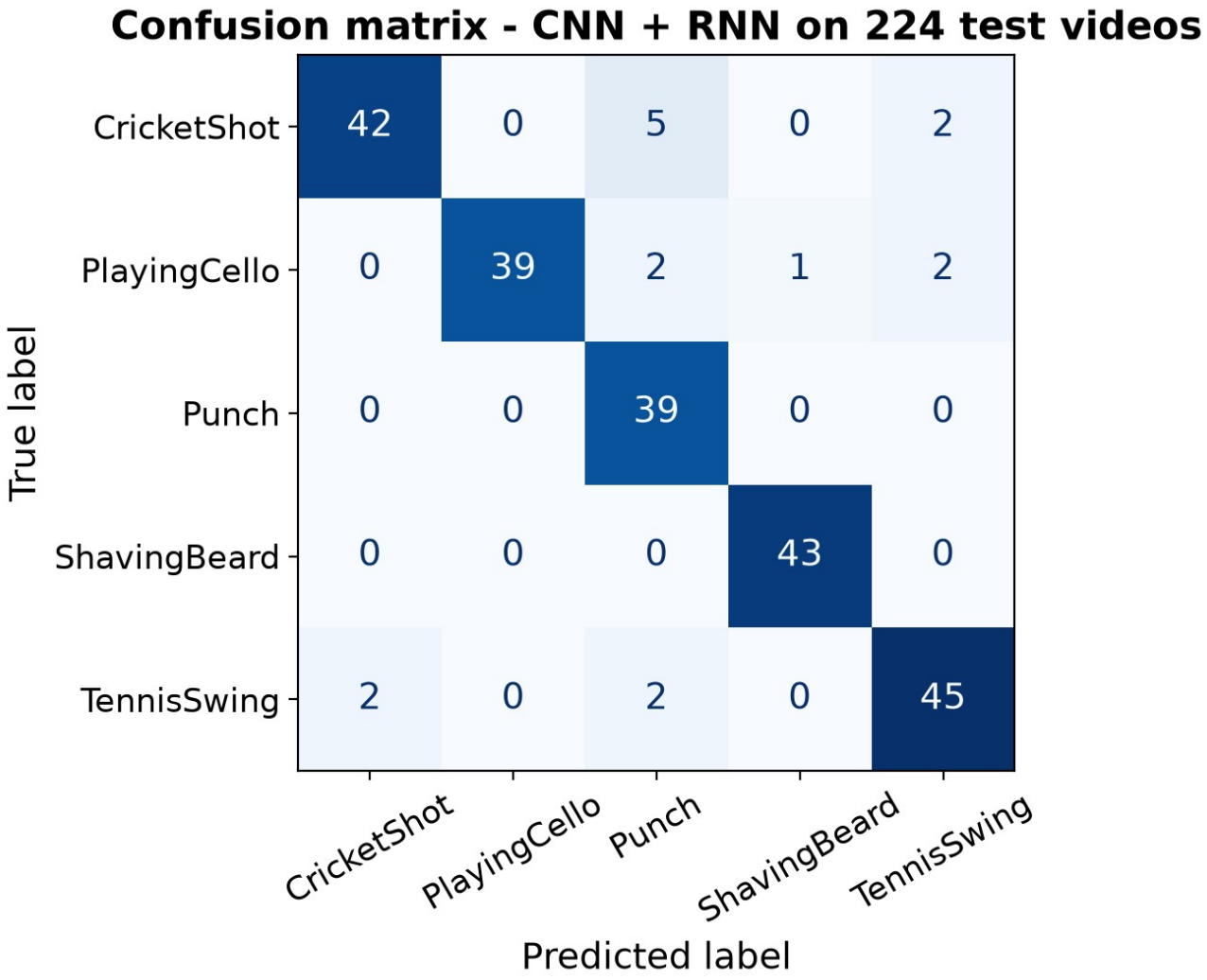
```
ConfusionMatrixDisplay(cm, display_labels=class_vocab).plot(ax=ax, cmap="Blues",
xticks_rotation=30, colorbar=False)
ax.set_title("Confusion matrix - CNN + RNN on 224 test videos")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
              precision    recall  f1-score   support

 CricketShot     0.9545    0.8571    0.9032        49
PlayingCello     1.0000    0.8864    0.9398        44
       Punch     0.8125    1.0000    0.8966        39
ShavingBeard     0.9773    1.0000    0.9885        43
 TennisSwing     0.9184    0.9184    0.9184        49

    accuracy                         0.9286       224
   macro avg     0.9325    0.9324    0.9293       224
weighted avg     0.9352    0.9286    0.9289       224
```



Step 5 : Single-video inference utilities

```
def prepare_single_video(frames):
    frames = frames[None, ...]
```

```
    frame_mask = np.zeros(shape=(1, MAX_SEQ_LENGTH), dtype="bool")
    frame_features = np.zeros(shape=(1, MAX_SEQ_LENGTH, NUM_FEATURES), dtype="float32")
    for i, batch in enumerate(frames):
        video_length = batch.shape[0]
        length = min(MAX_SEQ_LENGTH, video_length)
        for j in range(length):
            frame_features[i, j, :] = feature_extractor.predict(batch[None, j, :], verbose=0)
        frame_mask[i, :length] = 1          # 1 = not masked, 0 = masked
    return frame_features, frame_mask


def sequence_prediction(path):
    frames = load_video(os.path.join("test", path))
    frame_features, frame_mask = prepare_single_video(frames)
    probabilities = sequence_model.predict([frame_features, frame_mask], verbose=0)[0]
    for i in np.argsort(probabilities)[::-1]:
        print(f"  {class_vocab[i]}: {probabilities[i] * 100:5.2f}%")
    return frames, probabilities
```

Step 6 : Classify test videos end-to-end

```
np.random.seed(11)
picked = []
for tag in class_vocab:                                    # one random test video of every class
    picked.append(np.random.choice(test_df[test_df["tag"] == tag]["video_name"].values))

results = []
for test_video in picked:
    print(f"Test video path: {test_video}")
    frames, probabilities = sequence_prediction(test_video)
    results.append((test_video, frames, probabilities))
    print()
```

OUTPUT:

```
Test video path: v_CricketShot_g04_c05.avi
  CricketShot: 49.79%
  Punch: 34.54%
  TennisSwing: 11.34%
  ShavingBeard:  2.97%
  PlayingCello:  1.35%

Test video path: v_PlayingCello_g03_c03.avi
  ShavingBeard: 23.71%
  Punch: 20.45%
  CricketShot: 19.73%
  TennisSwing: 18.94%
  PlayingCello: 17.18%

Test video path: v_Punch_g06_c03.avi
  Punch: 52.42%
  CricketShot: 25.10%
  TennisSwing: 18.50%
  ShavingBeard:  2.86%
  PlayingCello:  1.11%

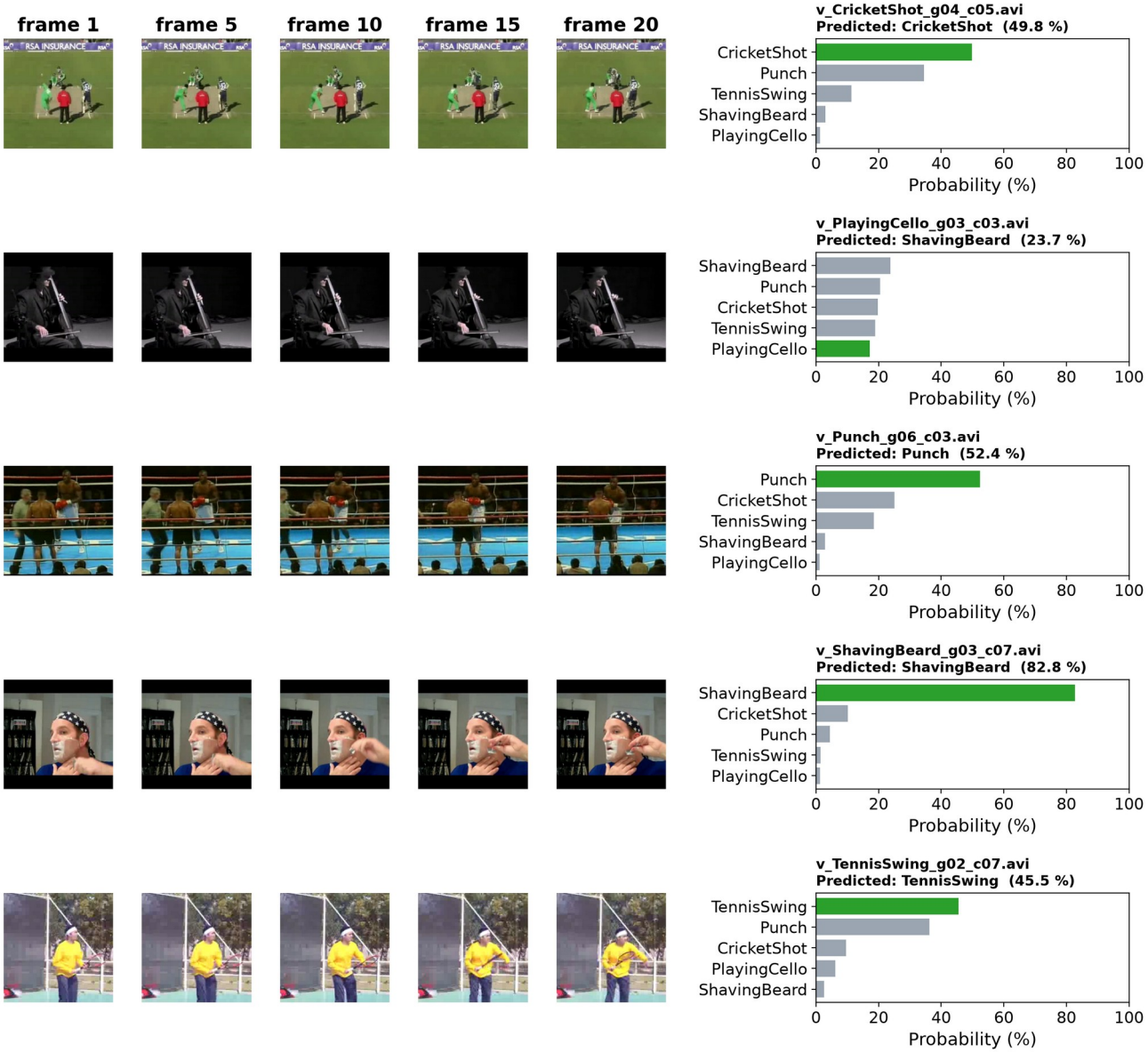
Test video path: v_ShavingBeard_g03_c07.avi
  ShavingBeard: 82.78%
  CricketShot: 10.11%
  Punch:  4.44%
  TennisSwing:  1.43%
  PlayingCello:  1.24%

Test video path: v_TennisSwing_g02_c07.avi
  TennisSwing: 45.51%
  Punch: 36.19%
  CricketShot:  9.54%
  PlayingCello:  6.15%
  ShavingBeard:  2.61%
```

Step 7 : Visualise the predictions

```
fig, axes = plt.subplots(len(results), 7, figsize=(13, 2.5 * len(results)),
                         gridspec_kw={"width_ratios": [1, 1, 1, 1, 1, 0.95, 2.4]})
for r, (name, frames, prob) in enumerate(results):
    for c, f in enumerate(np.linspace(0, min(len(frames), MAX_SEQ_LENGTH) - 1, 5, dtype=int)):
        axes[r, c].imshow(frames[f]); axes[r, c].axis("off")
        if r == 0: axes[r, c].set_title(f"frame {f + 1}")
    axes[r, 5].axis("off")
    order = np.argsort(prob)
    colors = ["#2ca02c" if class_vocab[i] == name.split("_")[1] else "#9aa5b1" for i in order]
    axes[r, 6].barh([class_vocab[i] for i in order], prob[order] * 100, color=colors)
    axes[r, 6].set_xlim(0, 100); axes[r, 6].set_xlabel("Probability (%)")
    pred = class_vocab[int(prob.argmax())]
    axes[r, 6].set_title(f"{name}\nPredicted: {pred}  ({prob.max() * 100:.1f} %)", fontsize=11,
loc="left")
fig.subplots_adjust(left=0.01, right=0.985, top=0.93, bottom=0.07, wspace=0.05, hspace=0.95)
plt.show()
```

OUTPUT:



Step 8 : Per-class accuracy summary

```
print(f"{'Class':<16}{'Videos':>8}{'Correct':>9}{'Accuracy':>11}")
for k, tag in enumerate(class_vocab):
    m = y_true == k
    print(f"{tag:<16}{m.sum():>8}{(y_pred[m] == k).sum():>9}{100 * (y_pred[m] ==
k).mean():>10.2f}%")
print("-" * 44)
print(f"{'Overall':<16}{len(y_true):>8}{(y_pred == y_true).sum():>9}{100 * (y_pred ==
y_true).mean():>10.2f}%")
```

OUTPUT:

```
Class            Videos  Correct   Accuracy
CricketShot          49       42     85.71%
PlayingCello         44       39     88.64%
Punch                39       39    100.00%
ShavingBeard         43       43    100.00%
TennisSwing          49       45     91.84%
-------------------------------------------
Overall             224      208     92.86%
```

RESULT:

Thus, the end-to-end CNN + RNN video classification pipeline was implemented successfully. The CNN extracted the spatial features of the frames and the GRU learned their temporal pattern. The complete model classified the 224 test videos with an overall accuracy of 92.86 % (macro F1-score 0.9293), and the predicted class probabilities of individual videos are shown in the output.

EX.NO.8	Vanilla GAN for Image Generation
DATE: 24.09.26	

AIM:

To implement a basic Vanilla Generative Adversarial Network (GAN) with a fully-connected generator and discriminator that learns to generate handwritten-digit images (MNIST) from random noise.

DESCRIPTION:

A Generative Adversarial Network consists of two neural networks that are trained against each other. The Generator G takes a random noise vector z and transforms it into a fake image. The Discriminator D receives an image and outputs the probability that it is real. D is trained to tell real images from fakes, while G is trained to produce images that D classifies as real. The two networks play the minimax game:

$$\min_G \max_D : \quad V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$$

In the Vanilla GAN both networks are built only from fully-connected (Dense) layers. The generator maps a 100-dimensional noise vector through Dense layers of 256, 512 and 1024 units (LeakyReLU and Batch Normalization) to 784 values with tanh activation, which are reshaped to a 28 × 28 image. The discriminator maps the 784 pixels through Dense layers of 512 and 256 units to one sigmoid output. Binary cross-entropy loss and the Adam optimizer (learning rate 0.0002, $\beta 1 = 0.5$) are used.

In every training step a batch of real images and a batch of generated images are scored by the discriminator. The discriminator loss rewards the label 1 for real images and the label 0 for fake images, while the generator loss rewards fake images that the discriminator labels 1. Each network is then updated with its own gradients using tf.GradientTape. When training is successful the discriminator accuracy approaches 50 %, which means that it can no longer distinguish real from generated digits.

ALGORITHM:

1. Load the MNIST images and scale the pixels to the range [−1, 1].
2. Build the generator (100 → 256 → 512 → 1024 → 784, tanh).
3. Build the discriminator (784 → 512 → 256 → 1, sigmoid).
4. Define the binary cross-entropy loss and the Adam optimizers of the generator and the discriminator.
5. For every batch: generate fake images from noise and let D score the real and the fake images.
6. Compute the discriminator loss (real → 1, fake → 0) and the generator loss (fake → 1) and update each network with its own gradients.
7. Repeat for 50 epochs and store the images generated from fixed noise at selected epochs.
8. Plot the loss and accuracy curves and display the generated digits.

ARCHITECTURE DIAGRAM:

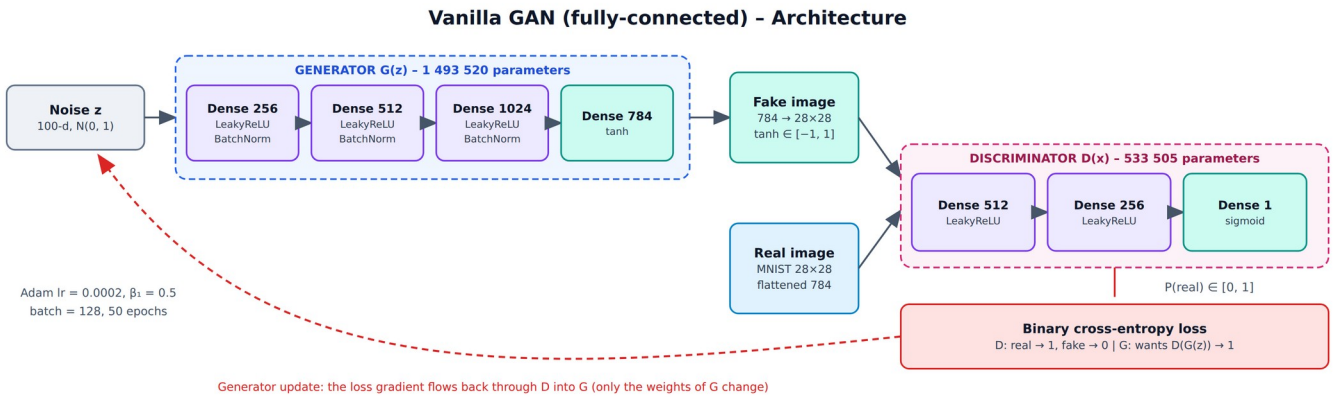


Fig. 8.1 Vanilla GAN architecture (generator, discriminator and training loop)

PROGRAM:

Step 1 : Import libraries

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42)
tf.random.set_seed(42)
print("TensorFlow version :", tf.__version__)
print("Keras version      :", keras.__version__)
```

OUTPUT:

```
TensorFlow version : 2.16.2
Keras version      : 3.15.1
```

Step 2 : Load and pre-process the MNIST images

```
(x_train, _), (_, _) = keras.datasets.mnist.load_data()

# scale pixels to [-1, 1] to match the tanh output of the generator
x_train = (x_train.astype("float32") - 127.5) / 127.5
x_train = x_train.reshape(-1, 784)

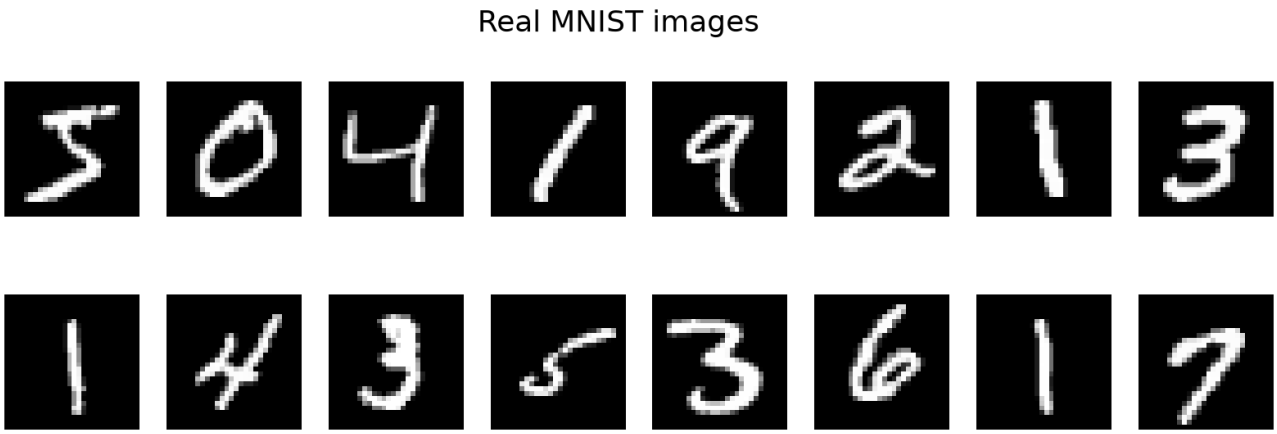
BATCH_SIZE = 128
dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000).batch(BATCH_SIZE,
drop_remainder=True)

print("Training images   :", x_train.shape)
print("Pixel range       :", x_train.min(), "to", x_train.max())
print("Batches per epoch :", len(dataset), "( batch size =", BATCH_SIZE, ")")

fig, axes = plt.subplots(2, 8, figsize=(10, 3.1))
for ax, img in zip(axes.ravel(), x_train[:16]):
    ax.imshow(img.reshape(28, 28), cmap="gray"); ax.axis("off")
plt.suptitle("Real MNIST images", fontsize=13)
plt.show()
```

OUTPUT:

```
Training images   : (60000, 784)
Pixel range       : -1.0 to 1.0
Batches per epoch : 468 ( batch size = 128 )
```



Step 3 : Generator network

```
LATENT_DIM = 100

def build_generator():
    model = keras.Sequential(name="generator")
    model.add(keras.Input(shape=(LATENT_DIM,)))
    model.add(layers.Dense(256))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(512))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(1024))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.BatchNormalization(momentum=0.8))
    model.add(layers.Dense(784, activation="tanh"))
    return model

generator = build_generator()
generator.summary()
```

OUTPUT:

Model: "generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	25,856
leaky_re_lu (LeakyReLU)	(None, 256)	0
batch_normalization (BatchNormalization)	(None, 256)	1,024
dense_1 (Dense)	(None, 512)	131,584
leaky_re_lu_1 (LeakyReLU)	(None, 512)	0
batch_normalization_1 (BatchNormalization)	(None, 512)	2,048
dense_2 (Dense)	(None, 1024)	525,312
leaky_re_lu_2 (LeakyReLU)	(None, 1024)	0
batch_normalization_2 (BatchNormalization)	(None, 1024)	4,096
dense_3 (Dense)	(None, 784)	803,600

Total params: 1,493,520 (5.70 MB)
Trainable params: 1,489,936 (5.68 MB)
Non-trainable params: 3,584 (14.00 KB)

Step 4 : Discriminator network

```
def build_discriminator():
    model = keras.Sequential(name="discriminator")
    model.add(keras.Input(shape=(784,)))
    model.add(layers.Dense(512))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Dense(256))
    model.add(layers.LeakyReLU(negative_slope=0.2))
    model.add(layers.Dense(1, activation="sigmoid"))
    return model

discriminator = build_discriminator()
discriminator.summary()
```

OUTPUT:

Model: "discriminator"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 512)	401,920
leaky_re_lu_3 (LeakyReLU)	(None, 512)	0
dense_5 (Dense)	(None, 256)	131,328
leaky_re_lu_4 (LeakyReLU)	(None, 256)	0
dense_6 (Dense)	(None, 1)	257

Total params: 533,505 (2.04 MB)
Trainable params: 533,505 (2.04 MB)
Non-trainable params: 0 (0.00 B)

Step 5 : Loss function and optimizers

```
cross_entropy = keras.losses.BinaryCrossentropy()

generator_optimizer = keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
discriminator_optimizer = keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
print("Loss      : Binary cross-entropy")
print("Optimizer : Adam (learning rate = 0.0002, beta_1 = 0.5) for the generator and the
discriminator")
```

OUTPUT:

```
Loss      : Binary cross-entropy
Optimizer : Adam (learning rate = 0.0002, beta_1 = 0.5) for the generator and the discriminator
```

Step 6 : Training step

```
@tf.function
def train_step(real_imgs):
    noise = tf.random.normal((tf.shape(real_imgs)[0], LATENT_DIM))
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_imgs = generator(noise, training=True)
        real_output = discriminator(real_imgs, training=True)
        fake_output = discriminator(fake_imgs, training=True)
        # discriminator: real -> 1, fake -> 0        generator: fake -> 1
        d_loss = 0.5 * (cross_entropy(tf.ones_like(real_output), real_output)
                        + cross_entropy(tf.zeros_like(fake_output), fake_output))
        g_loss = cross_entropy(tf.ones_like(fake_output), fake_output)
    d_grads = disc_tape.gradient(d_loss, discriminator.trainable_variables)
    g_grads = gen_tape.gradient(g_loss, generator.trainable_variables)
    discriminator_optimizer.apply_gradients(zip(d_grads, discriminator.trainable_variables))
    generator_optimizer.apply_gradients(zip(g_grads, generator.trainable_variables))
    d_acc = 0.5 * (tf.reduce_mean(tf.cast(real_output > 0.5, tf.float32))
                   + tf.reduce_mean(tf.cast(fake_output < 0.5, tf.float32)))
    return d_loss, d_acc, g_loss

print("Training step defined: the discriminator and the generator are updated with their own
gradients.")
```

OUTPUT:

```
Training step defined: the discriminator and the generator are updated with their own gradients.
```

Step 7 : Train the Vanilla GAN

```
EPOCHS = 50
fixed_noise = tf.random.normal((16, LATENT_DIM))
history = {"d_loss": [], "d_acc": [], "g_loss": []}
snapshots = {}

for epoch in range(1, EPOCHS + 1):
    d_l, d_a, g_l = [], [], []
    for real_batch in dataset:
        d_loss, d_acc, g_loss = train_step(real_batch)
        d_l.append(float(d_loss)); d_a.append(float(d_acc)); g_l.append(float(g_loss))
    history["d_loss"].append(np.mean(d_l)); history["d_acc"].append(np.mean(d_a));
history["g_loss"].append(np.mean(g_l))
    if epoch in (1, 10, 25, 50):
        snapshots[epoch] = generator(fixed_noise, training=False).numpy()
    if epoch == 1 or epoch % 5 == 0:
        print(f"Epoch {epoch:>2}/{EPOCHS}  [D loss: {history['d_loss'][-1]:.4f}, acc: {100 *
history['d_acc'][-1]:.2f}%]  [G loss: {history['g_loss'][-1]:.4f}]")
print("Vanilla GAN training completed.")
```

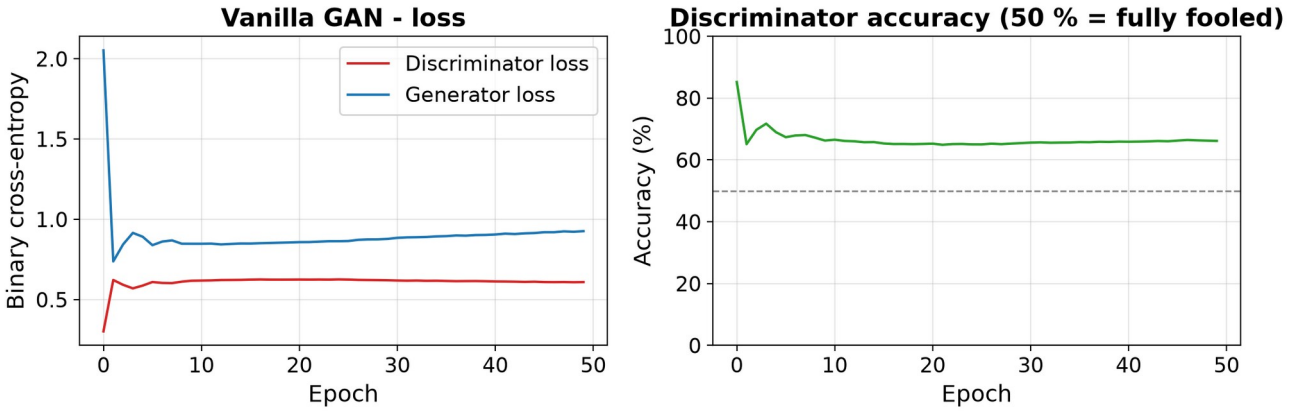
OUTPUT:

```
Epoch  1/50  [D loss: 0.3024, acc: 85.21%]  [G loss: 2.0525]
Epoch  5/50  [D loss: 0.5873, acc: 68.99%]  [G loss: 0.8919]
Epoch 10/50  [D loss: 0.6178, acc: 66.27%]  [G loss: 0.8480]
Epoch 15/50  [D loss: 0.6230, acc: 65.77%]  [G loss: 0.8496]
Epoch 20/50  [D loss: 0.6246, acc: 65.18%]  [G loss: 0.8560]
Epoch 25/50  [D loss: 0.6262, acc: 65.03%]  [G loss: 0.8639]
Epoch 30/50  [D loss: 0.6205, acc: 65.44%]  [G loss: 0.8783]
Epoch 35/50  [D loss: 0.6179, acc: 65.64%]  [G loss: 0.8939]
Epoch 40/50  [D loss: 0.6146, acc: 65.93%]  [G loss: 0.9029]
Epoch 45/50  [D loss: 0.6117, acc: 66.05%]  [G loss: 0.9146]
Epoch 50/50  [D loss: 0.6092, acc: 66.17%]  [G loss: 0.9263]
Vanilla GAN training completed.
```

Step 8 : Training curves

```
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
ax[0].plot(history["d_loss"], label="Discriminator loss", color="#d62728")
ax[0].plot(history["g_loss"], label="Generator loss", color="#1f77b4")
ax[0].set_title("Vanilla GAN - loss"); ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Binary cross-
entropy"); ax[0].legend(); ax[0].grid(alpha=.3)
ax[1].plot(np.array(history["d_acc"]) * 100, color="#2ca02c")
ax[1].axhline(50, ls="--", color="grey", lw=1)
ax[1].set_title("Discriminator accuracy (50 % = fully fooled)"); ax[1].set_xlabel("Epoch");
ax[1].set_ylabel("Accuracy (%)"); ax[1].set_ylim(0, 100); ax[1].grid(alpha=.3)
plt.tight_layout()
plt.show()
```

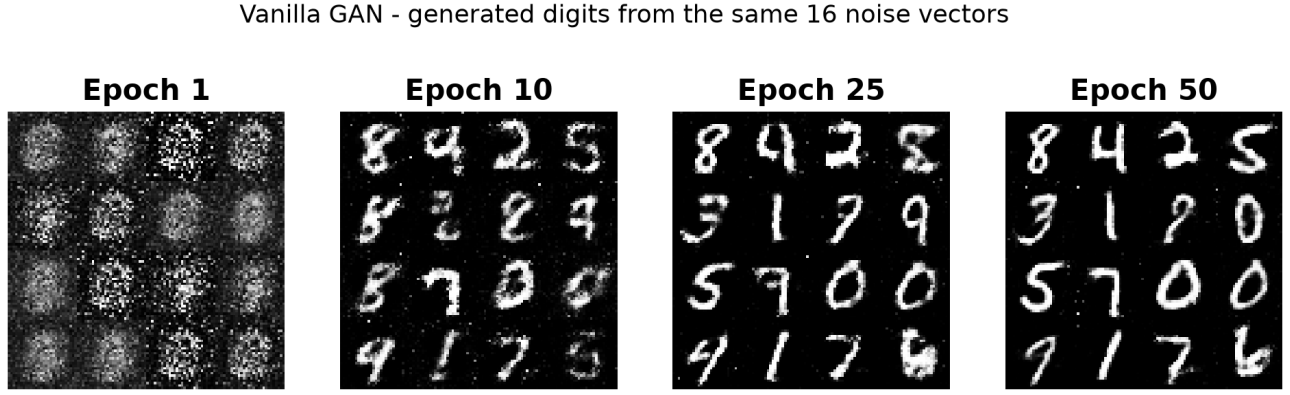
OUTPUT:



Step 9 : Images generated during training

```
fig, axes = plt.subplots(1, 4, figsize=(12, 3.7))
for ax, ep in zip(axes, snapshots):
    grid = snapshots[ep].reshape(4, 4, 28, 28).transpose(0, 2, 1, 3).reshape(4 * 28, 4 * 28)
    ax.imshow(grid, cmap="gray"); ax.set_title(f"Epoch {ep}"); ax.axis("off")
plt.suptitle("Vanilla GAN - generated digits from the same 16 noise vectors", fontsize=13)
plt.show()
```

OUTPUT:



Step 10 : Generate new images from random noise

```
noise = tf.random.normal((25, LATENT_DIM))
gen_imgs = generator(noise, training=False).numpy().reshape(-1, 28, 28)
gen_imgs = 0.5 * gen_imgs + 0.5                        # back to [0, 1]

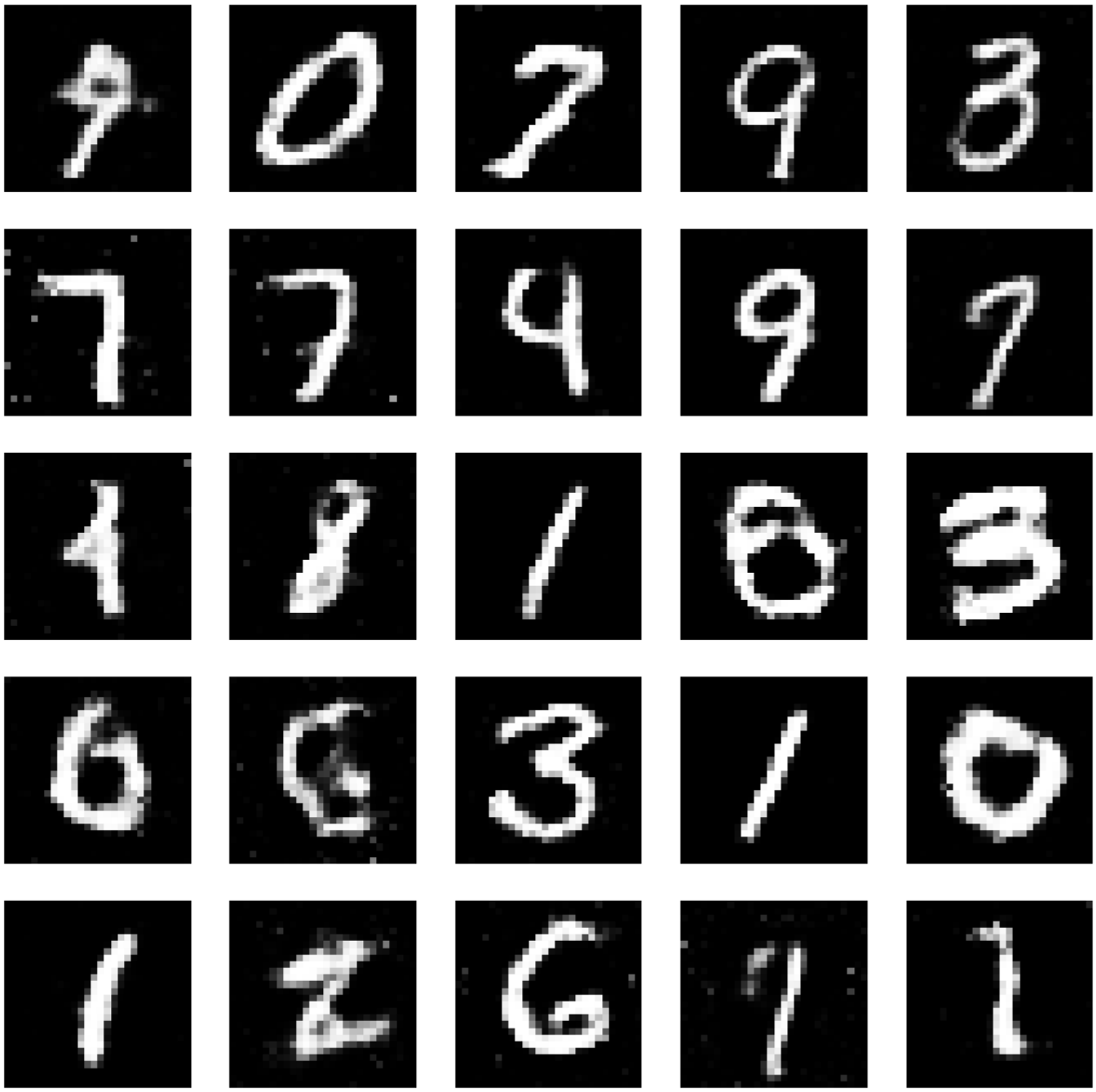
print("Noise vector shape     :", tuple(noise.shape))
print("Generated image batch  :", gen_imgs.shape)
print("Generated pixel range  : %.3f to %.3f" % (gen_imgs.min(), gen_imgs.max()))
print("Discriminator score on generated images (mean) : %.4f" % discriminator(gen_imgs.reshape(-
1, 784) * 2 - 1, training=False).numpy().mean())

fig, axes = plt.subplots(5, 5, figsize=(6.5, 6.5))
for ax, img in zip(axes.ravel(), gen_imgs):
    ax.imshow(img, cmap="gray"); ax.axis("off")
plt.suptitle("Images generated by the trained Vanilla GAN", fontsize=12)
plt.show()
```

OUTPUT:

```
Noise vector shape     : (25, 100)
Generated image batch  : (25, 28, 28)
Generated pixel range  : 0.000 to 1.000
Discriminator score on generated images (mean) : 0.4341
```

Images generated by the trained Vanilla GAN



RESULT:

Thus, the Vanilla GAN was implemented successfully. After 50 epochs of training the generator produced new handwritten-digit images from random noise (final discriminator loss 0.6092, accuracy 66.17 %, generator loss 0.9263).

EX.NO.9	DCGAN (Deep Convolutional GAN)
DATE: 24.09.26	

AIM:

To implement a Deep Convolutional GAN (DCGAN) that uses transposed-convolution layers in the generator and strided-convolution layers in the discriminator to generate realistic handwritten-digit images from random noise.

DESCRIPTION:

The DCGAN (Radford et al., 2015) replaces the fully-connected layers of the Vanilla GAN by convolutional layers, which exploit the spatial structure of images. Its main design rules are:

- Replace pooling layers by strided convolutions (discriminator) and transposed convolutions (generator).
- Use Batch Normalization in both networks (except the output of the generator and the input of the discriminator).
- Remove fully-connected hidden layers; use ReLU / LeakyReLU activations and tanh at the generator output.

Generator:

The 100-dimensional noise vector is projected by a Dense layer to $7 \times 7 \times 256$ and reshaped. Three Conv2DTranspose layers (128 filters, stride 1 → 64 filters, stride 2 → 1 filter, stride 2) up-sample it step by step to 14×14 and finally to a $28 \times 28 \times 1$ image with tanh activation.

Discriminator:

A CNN classifier: two Conv2D layers (64 and 128 filters, 5×5, stride 2) with LeakyReLU and Dropout (0.3) reduce the image to $7 \times 7 \times 128$, which is flattened and passed to one Dense unit that gives the real/fake logit.

Training:

The discriminator loss is the sum of the cross-entropy on real images (target 1) and fake images (target 0); the generator loss is the cross-entropy of the fake images against the target 1. Both networks use the Adam optimizer (learning rate 2e-4, $\beta 1 = 0.5$) as recommended in the DCGAN paper. The training step is compiled with tf.function and uses two GradientTapes, one for each network.

ALGORITHM:

1. Load the MNIST dataset, normalise the pixels to $[-1, 1]$ and create shuffled batches of 128 images.
2. Build the generator with Dense, Reshape and three Conv2DTranspose layers.
3. Build the discriminator with two strided Conv2D layers, Dropout, Flatten and Dense.
4. Define the generator loss, the discriminator loss and the Adam optimizers.
5. In every training step generate fake images, score real and fake images and compute both losses.
6. Update the generator and the discriminator with their gradients.
7. Train for the chosen number of epochs and save the images generated from a fixed seed after selected epochs.
8. Plot the training losses and display the final generated digits.

ARCHITECTURE DIAGRAM:

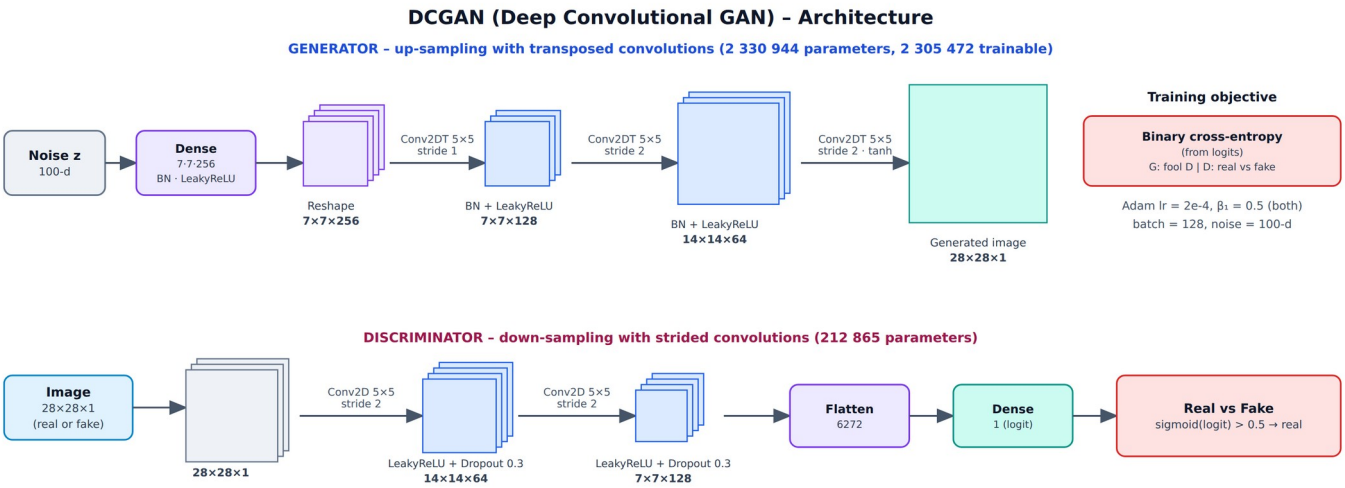


Fig. 9.1 DCGAN generator and discriminator architecture

PROGRAM:

Step 1 : Import libraries

```
import time
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers

tf.random.set_seed(42)
np.random.seed(42)
print("TensorFlow version :", tf.__version__)
```

OUTPUT:

```
TensorFlow version : 2.16.2
```

Step 2 : Load and prepare the MNIST dataset

```
(train_images, _), (_, _) = keras.datasets.mnist.load_data()
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype("float32")
train_images = (train_images - 127.5) / 127.5          # normalise the images to [-1, 1]

BUFFER_SIZE = 60000
BATCH_SIZE = 128
train_dataset =
tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

print("Training images :", train_images.shape)
print("Batches per epoch :", len(train_dataset), "( batch size =", BATCH_SIZE, ")")
```

OUTPUT:

```
Training images : (60000, 28, 28, 1)
Batches per epoch : 469 ( batch size = 128 )
```

Step 3 : Generator model

```
def make_generator_model():
    model = keras.Sequential(name="generator")
    model.add(keras.Input(shape=(100,)))
```

```
    model.add(layers.Dense(7 * 7 * 256, use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Reshape((7, 7, 256)))

    model.add(layers.Conv2DTranspose(128, (5, 5), strides=(1, 1), padding="same",
use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64, (5, 5), strides=(2, 2), padding="same", use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding="same", use_bias=False,
activation="tanh"))
    return model

generator = make_generator_model()
generator.summary()
```

OUTPUT:

Model: "generator"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 12544)	1,254,400
batch_normalization (BatchNormalization)	(None, 12544)	50,176
leaky_re_lu (LeakyReLU)	(None, 12544)	0
reshape (Reshape)	(None, 7, 7, 256)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 128)	819,200
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 128)	512
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 64)	204,800
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
leaky_re_lu_2 (LeakyReLU)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	1,600

Total params: 2,330,944 (8.89 MB)
Trainable params: 2,305,472 (8.79 MB)
Non-trainable params: 25,472 (99.50 KB)

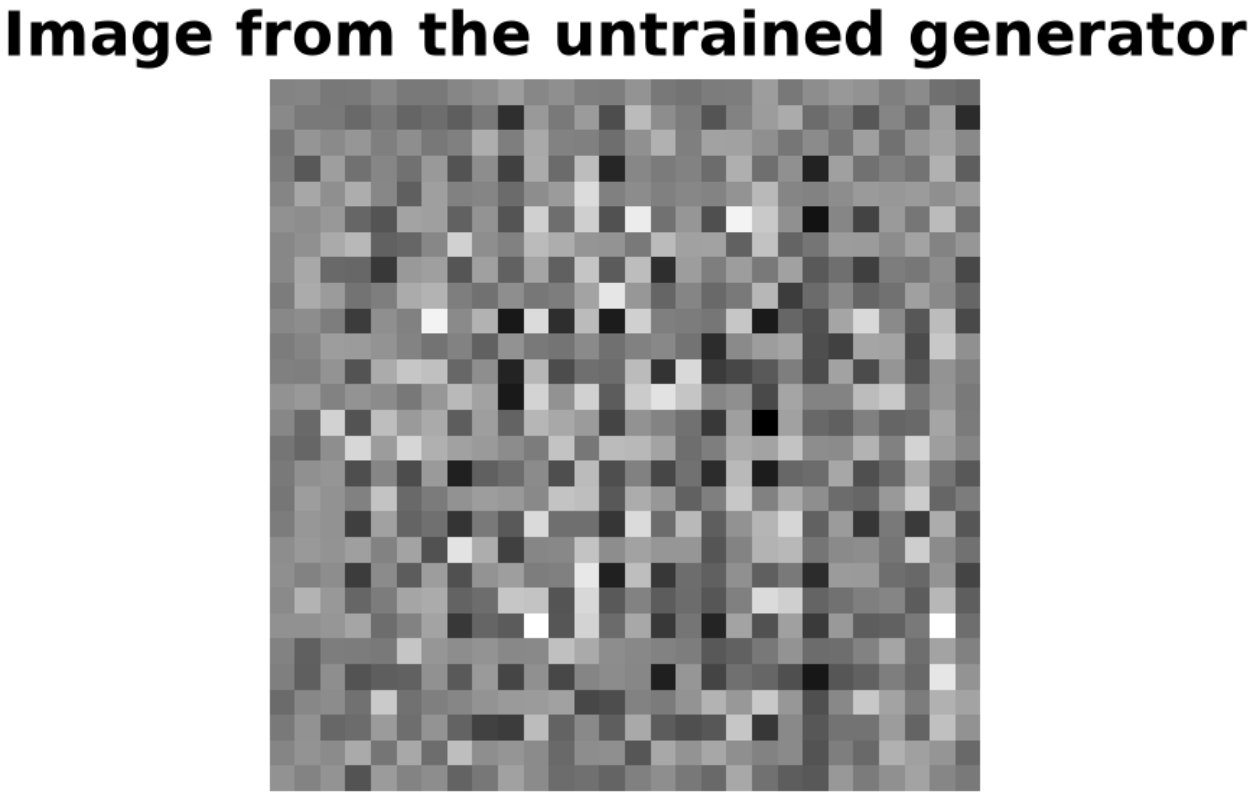
Step 4 : Untrained generator output

```
noise = tf.random.normal([1, 100])
generated_image = generator(noise, training=False)
print("Generated image shape :", generated_image.shape)
```

```
plt.figure(figsize=(3.2, 3.2))
plt.imshow(generated_image[0, :, :, 0], cmap="gray")
plt.title("Image from the untrained generator"); plt.axis("off")
plt.show()
```

OUTPUT:

```
Generated image shape : (1, 28, 28, 1)
```



Step 5 : Discriminator model

```
def make_discriminator_model():
    model = keras.Sequential(name="discriminator")
    model.add(keras.Input(shape=(28, 28, 1)))
    model.add(layers.Conv2D(64, (5, 5), strides=(2, 2), padding="same"))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, (5, 5), strides=(2, 2), padding="same"))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1))
    return model

discriminator = make_discriminator_model()
decision = discriminator(generated_image)
print("Discriminator decision on the untrained fake image :", decision.numpy())
discriminator.summary()
```

OUTPUT:

```
Discriminator decision on the untrained fake image : [[0.00270408]]
Model: "discriminator"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_3 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_4 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense_1 (Dense)	(None, 1)	6,273

```
Total params: 212,865 (831.50 KB)
Trainable params: 212,865 (831.50 KB)
Non-trainable params: 0 (0.00 B)
```

Step 6 : Loss functions and optimizers

```
cross_entropy = keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)
discriminator_optimizer = keras.optimizers.Adam(learning_rate=2e-4, beta_1=0.5)
print("Loss  : Binary cross-entropy (from logits)")
print("Optim : Adam, learning rate = 2e-4, beta_1 = 0.5 (generator and discriminator)")
```

OUTPUT:

```
Loss  : Binary cross-entropy (from logits)
Optim : Adam, learning rate = 2e-4, beta_1 = 0.5 (generator and discriminator)
```

Step 7 : Training step and training loop

```
EPOCHS = 15
noise_dim = 100
num_examples_to_generate = 16
seed = tf.random.normal([num_examples_to_generate, noise_dim])

@tf.function
def train_step(images):
    noise = tf.random.normal([tf.shape(images)[0], noise_dim])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)
        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)
        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)
```

```
    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)
    generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))
    return gen_loss, disc_loss

history = {"gen": [], "disc": []}
snapshots = {}
SNAP_EPOCHS = sorted({1, EPOCHS // 6, EPOCHS // 2, EPOCHS})

def train(dataset, epochs):
    for epoch in range(1, epochs + 1):
        start = time.time()
        g_l, d_l = [], []
        for image_batch in dataset:
            g, d = train_step(image_batch)
            g_l.append(float(g)); d_l.append(float(d))
        history["gen"].append(np.mean(g_l)); history["disc"].append(np.mean(d_l))
        if epoch in SNAP_EPOCHS:
            snapshots[epoch] = generator(seed, training=False).numpy()
        print("Epoch {:>2}/{}  gen_loss: {:.4f}  disc_loss: {:.4f}  - Time for epoch {} is {:.2f}
sec".format(epoch, epochs, history["gen"][-1], history["disc"][-1], epoch, time.time() - start))

train(train_dataset, EPOCHS)
print("DCGAN training completed.")
```

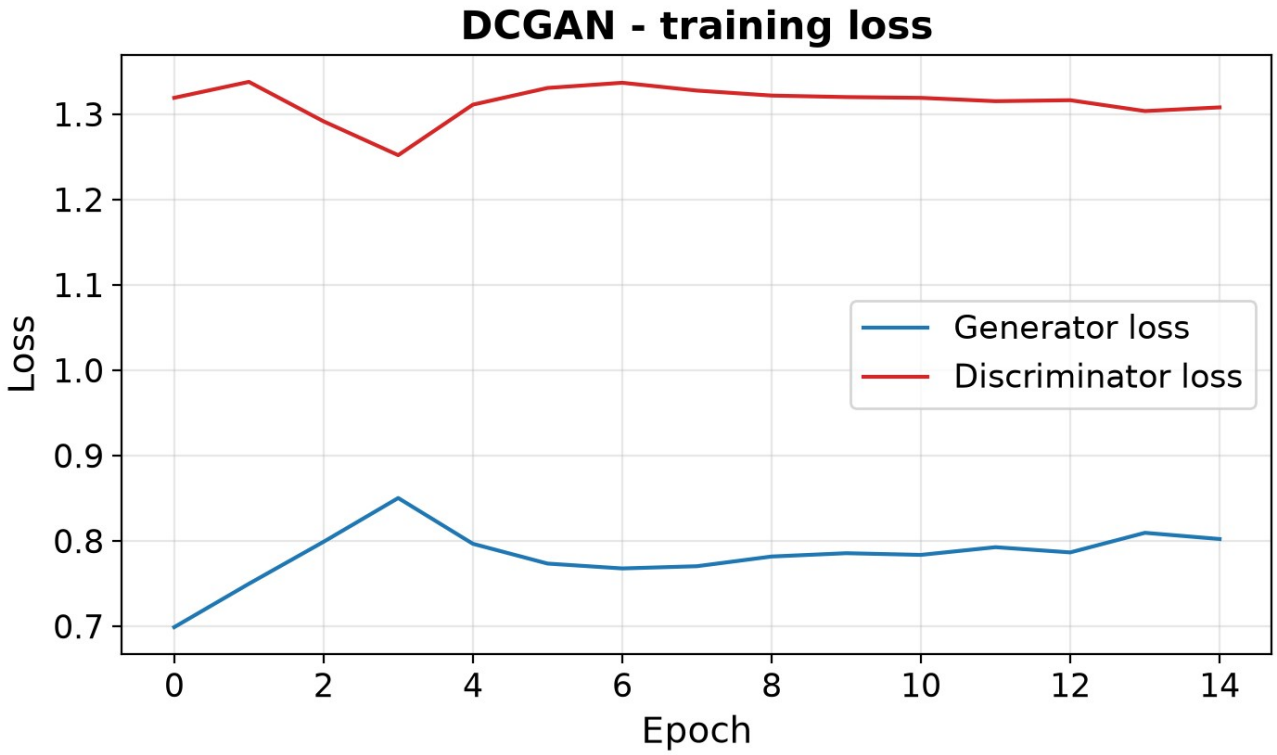
OUTPUT:

```
Epoch  1/15  gen_loss: 0.6989  disc_loss: 1.3193  - Time for epoch 1 is 172.19 sec
Epoch  2/15  gen_loss: 0.7498  disc_loss: 1.3379  - Time for epoch 2 is 169.73 sec
Epoch  3/15  gen_loss: 0.7990  disc_loss: 1.2917  - Time for epoch 3 is 168.42 sec
Epoch  4/15  gen_loss: 0.8503  disc_loss: 1.2521  - Time for epoch 4 is 167.96 sec
Epoch  5/15  gen_loss: 0.7966  disc_loss: 1.3112  - Time for epoch 5 is 170.41 sec
Epoch  6/15  gen_loss: 0.7735  disc_loss: 1.3308  - Time for epoch 6 is 166.72 sec
Epoch  7/15  gen_loss: 0.7678  disc_loss: 1.3369  - Time for epoch 7 is 201.96 sec
Epoch  8/15  gen_loss: 0.7704  disc_loss: 1.3277  - Time for epoch 8 is 167.51 sec
Epoch  9/15  gen_loss: 0.7817  disc_loss: 1.3219  - Time for epoch 9 is 155.27 sec
Epoch 10/15  gen_loss: 0.7856  disc_loss: 1.3201  - Time for epoch 10 is 173.95 sec
Epoch 11/15  gen_loss: 0.7837  disc_loss: 1.3192  - Time for epoch 11 is 168.88 sec
Epoch 12/15  gen_loss: 0.7927  disc_loss: 1.3153  - Time for epoch 12 is 169.29 sec
Epoch 13/15  gen_loss: 0.7866  disc_loss: 1.3164  - Time for epoch 13 is 172.16 sec
Epoch 14/15  gen_loss: 0.8095  disc_loss: 1.3037  - Time for epoch 14 is 173.79 sec
Epoch 15/15  gen_loss: 0.8022  disc_loss: 1.3080  - Time for epoch 15 is 201.95 sec
DCGAN training completed.
```

Step 8 : Training curves

```
plt.figure(figsize=(8, 4.2))
plt.plot(history["gen"], label="Generator loss", color="#1f77b4")
plt.plot(history["disc"], label="Discriminator loss", color="#d62728")
plt.title("DCGAN - training loss"); plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.legend();
plt.grid(alpha=.3)
plt.show()
```

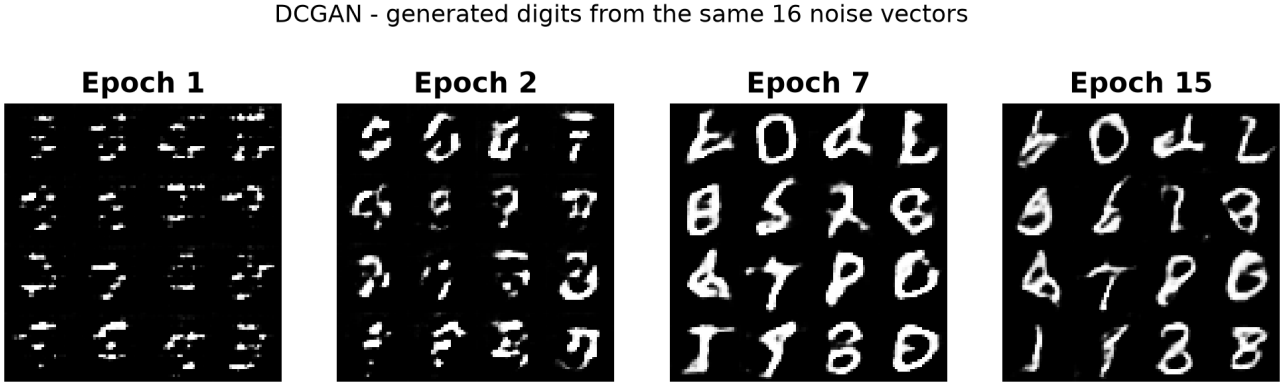
OUTPUT:



Step 9 : Images generated during training

```
fig, axes = plt.subplots(1, len(snapshots), figsize=(3.1 * len(snapshots), 3.7))
for ax, ep in zip(axes, snapshots):
    grid = ((snapshots[ep][..., 0] + 1) / 2).reshape(4, 4, 28, 28).transpose(0, 2, 1,
3).reshape(4 * 28, 4 * 28)
    ax.imshow(grid, cmap="gray"); ax.set_title(f"Epoch {ep}"); ax.axis("off")
plt.suptitle("DCGAN - generated digits from the same 16 noise vectors", fontsize=13)
plt.show()
```

OUTPUT:

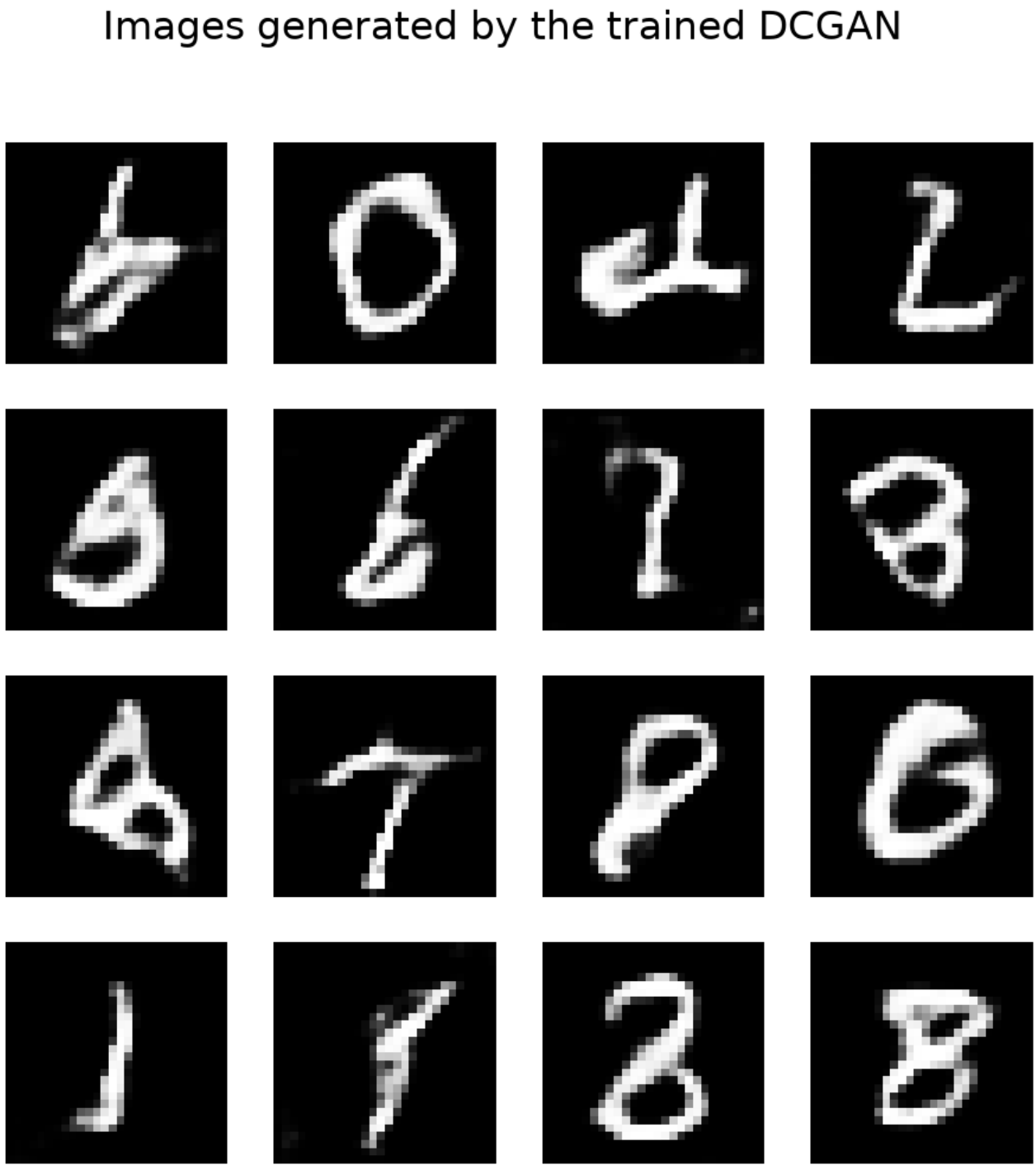


Step 10 : Generate the final images

```
predictions = generator(seed, training=False).numpy()
fig = plt.figure(figsize=(5.6, 5.6))
for i in range(predictions.shape[0]):
    plt.subplot(4, 4, i + 1)
    plt.imshow(predictions[i, :, :, 0] * 127.5 + 127.5, cmap="gray")
    plt.axis("off")
plt.suptitle("Images generated by the trained DCGAN", fontsize=12)
plt.show()
print("Final discriminator score on generated images (logit mean) : %.4f" %
discriminator(generator(seed, training=False), training=False).numpy().mean())
```

OUTPUT:

```
Final discriminator score on generated images (logit mean) : -0.1927
```



RESULT:

Thus, the DCGAN was implemented successfully with transposed-convolution up-sampling in the generator and strided-convolution down-sampling in the discriminator. After 15 epochs the generator produced clear, well-formed handwritten digits from random noise, much sharper than those of the fully-connected Vanilla GAN.

EX.NO.10	Conditional GAN (cGAN)
DATE: 24.09.26	

AIM:

To implement a Conditional GAN (cGAN) in which the class label is given to both the generator and the discriminator together with the input, so that the generator produces images of the digit class that is requested.

DESCRIPTION:

A Conditional GAN (Mirza and Osindero, 2014) extends the GAN by feeding an extra piece of information y – here the digit label 0–9 – to both networks. The generator therefore learns G(z | y), an image of class y, and the discriminator learns D(x | y), the probability that x is a real image of class y. The objective becomes:

$$\min_G \max_D : \; V(D, G) = E[\log D(x \mid y)] + E[\log(1 - D(G(z \mid y) \mid y))]$$

Conditioning the networks:

- **Generator –** the label is passed through an Embedding layer (10 × 50), a Dense layer and a reshape to a 7 × 7 map, which is concatenated with the 7 × 7 × 128 map produced from the noise. Three Conv2DTranspose layers (64 filters stride 1, 32 filters stride 2, 1 filter stride 2 with tanh) with Batch Normalization up-sample it to the 28 × 28 image.
- **Discriminator –** the label is embedded, scaled up to a 28 × 28 map and concatenated with the image as a second channel; two strided 5 × 5 Conv2D layers (64 and 128 filters) with LeakyReLU and Dropout (0.3), Flatten and a sigmoid Dense layer give the decision.

Because the label controls the content, the user can choose the digit to be generated. As an independent check, a small CNN classifier trained on real MNIST digits is used to verify that the generated digits really belong to the requested classes.

ALGORITHM:

1. Load the MNIST images and labels and scale the pixels to [−1, 1].
2. Build the conditional discriminator with inputs (image, label).
3. Build the conditional generator with inputs (noise, label).
4. Define the binary cross-entropy loss and the Adam optimizers of the generator and the discriminator.
5. For each batch: generate images for random class labels and let D score the real (image, label) pairs and the generated pairs.
6. Update D so that it separates real from generated pairs, and update G so that D accepts its images for the requested labels (tf.GradientTape).
7. Repeat for the chosen number of epochs and save samples of all ten classes at selected epochs.
8. Generate one digit for every label 0–9 and measure the conditional accuracy with a CNN classifier.

ARCHITECTURE DIAGRAM:

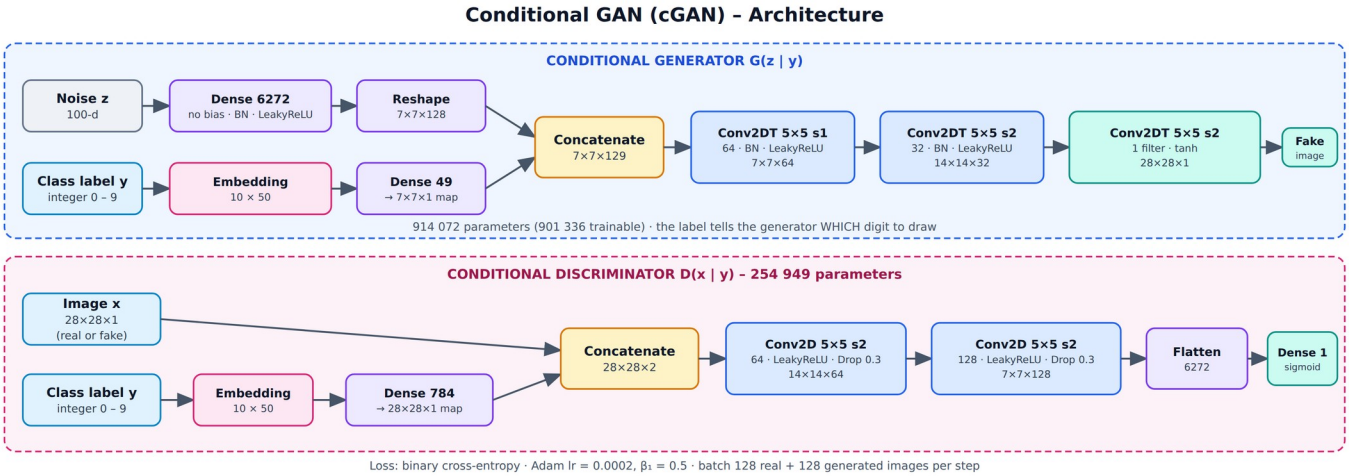


Fig. 10.1 Conditional GAN architecture (generator and discriminator)

PROGRAM:

Step 1 : Import libraries

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42)
tf.random.set_seed(42)
print("TensorFlow version :", tf.__version__)
```

OUTPUT:

```
TensorFlow version : 2.16.2
```

Step 2 : Load the MNIST images with their class labels

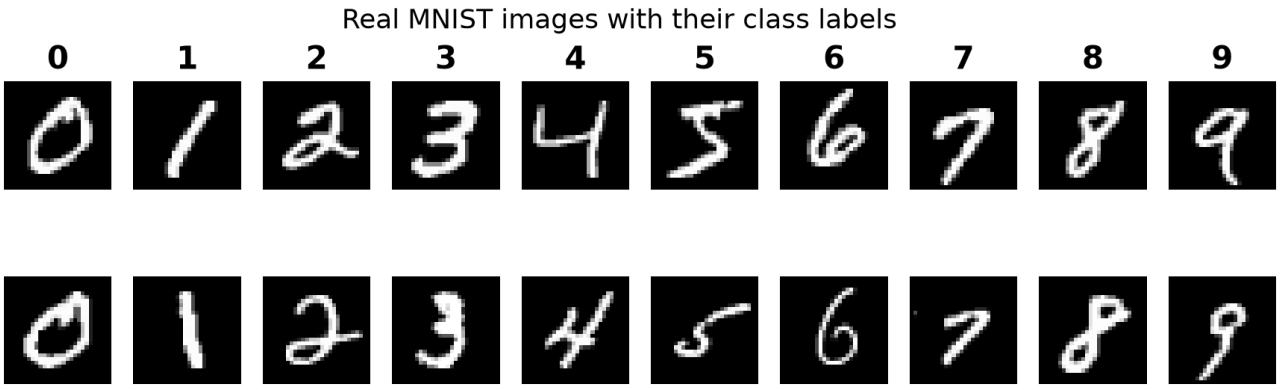
```
(x_train, y_train), (_, _) = keras.datasets.mnist.load_data()
x_train = np.expand_dims(x_train.astype("float32"), -1)
x_train = (x_train - 127.5) / 127.5                    # scale to [-1, 1]

LATENT_DIM = 100
NUM_CLASSES = 10
print("Images :", x_train.shape, "| Labels :", y_train.shape)
print("Classes:", np.unique(y_train))

fig, axes = plt.subplots(2, 10, figsize=(11, 3.1))
for d in range(10):
    for r in range(2):
        axes[r, d].imshow(x_train[np.where(y_train == d)[0][r], :, :, 0], cmap="gray"); axes[r,
d].axis("off")
    axes[0, d].set_title(f"{d}")
plt.suptitle("Real MNIST images with their class labels", fontsize=13)
plt.show()
```

OUTPUT:

```
Images : (60000, 28, 28, 1) | Labels : (60000,)
Classes: [0 1 2 3 4 5 6 7 8 9]
```



Step 3 : Conditional discriminator

```
def build_discriminator(in_shape=(28, 28, 1), n_classes=NUM_CLASSES):
    in_label = keras.Input(shape=(1,))
    li = layers.Embedding(n_classes, 50)(in_label)             # label -> 50-d vector
    li = layers.Dense(in_shape[0] * in_shape[1])(li)           # scale up to image size
    li = layers.Reshape((in_shape[0], in_shape[1], 1))(li)
    in_image = keras.Input(shape=in_shape)
    merge = layers.Concatenate()([in_image, li])               # image + label channel (28x28x2)
    x = layers.Conv2D(64, (5, 5), strides=(2, 2), padding="same")(merge)
    x = layers.LeakyReLU(negative_slope=0.2)(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Conv2D(128, (5, 5), strides=(2, 2), padding="same")(x)
    x = layers.LeakyReLU(negative_slope=0.2)(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Flatten()(x)
    out = layers.Dense(1, activation="sigmoid")(x)
    return keras.Model([in_image, in_label], out, name="discriminator")

discriminator = build_discriminator()
discriminator.summary()
```

OUTPUT:

Model: "discriminator"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 1)	0	-
embedding (Embedding)	(None, 1, 50)	500	input_layer[0][0]
dense (Dense)	(None, 1, 784)	39,984	embedding[0][0]
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0	-
reshape (Reshape)	(None, 28, 28, 1)	0	dense[0][0]
concatenate (Concatenate)	(None, 28, 28, 2)	0	input_layer_1[0]… reshape[0][0]
conv2d (Conv2D)	(None, 14, 14, 64)	3,264	concatenate[0][0]
leaky_re_lu (LeakyReLU)	(None, 14, 14, 64)	0	conv2d[0][0]
dropout (Dropout)	(None, 14, 14, 64)	0	leaky_re_lu[0][0]
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928	dropout[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 7, 7, 128)	0	conv2d_1[0][0]
dropout_1 (Dropout)	(None, 7, 7, 128)	0	leaky_re_lu_1[0]…
flatten (Flatten)	(None, 6272)	0	dropout_1[0][0]
dense_1 (Dense)	(None, 1)	6,273	flatten[0][0]

Total params: 254,949 (995.89 KB)
Trainable params: 254,949 (995.89 KB)
Non-trainable params: 0 (0.00 B)

Step 4 : Conditional generator

```
def build_generator(latent_dim=LATENT_DIM, n_classes=NUM_CLASSES):
    in_label = keras.Input(shape=(1,))
    li = layers.Embedding(n_classes, 50)(in_label)
    li = layers.Dense(7 * 7)(li)
    li = layers.Reshape((7, 7, 1))(li)                          # label as a 7x7 feature map
    in_lat = keras.Input(shape=(latent_dim,))
    gen = layers.Dense(7 * 7 * 128, use_bias=False)(in_lat)
    gen = layers.BatchNormalization()(gen)
    gen = layers.LeakyReLU(negative_slope=0.2)(gen)
    gen = layers.Reshape((7, 7, 128))(gen)
    merge = layers.Concatenate()([gen, li])                     # noise map + label map (7x7x129)
    x = layers.Conv2DTranspose(64, (5, 5), strides=(1, 1), padding="same", use_bias=False)(merge)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(negative_slope=0.2)(x)                 # 7x7x64
    x = layers.Conv2DTranspose(32, (5, 5), strides=(2, 2), padding="same", use_bias=False)(x)
    x = layers.BatchNormalization()(x)
```

```
    x = layers.LeakyReLU(negative_slope=0.2)(x)                # 14x14x32
    out = layers.Conv2DTranspose(1, (5, 5), strides=(2, 2), padding="same", activation="tanh")(x)
# 28x28x1
    return keras.Model([in_lat, in_label], out, name="generator")

generator = build_generator()
generator.summary()
```

OUTPUT:

Model: "generator"

Layer (type)	Output Shape	Param #	Connected to
input_layer_3 (InputLayer)	(None, 100)	0	-
dense_3 (Dense)	(None, 6272)	627,200	input_layer_3[0]…
input_layer_2 (InputLayer)	(None, 1)	0	-
batch_normalization (BatchNormalizatio…	(None, 6272)	25,088	dense_3[0][0]
embedding_1 (Embedding)	(None, 1, 50)	500	input_layer_2[0]…
leaky_re_lu_2 (LeakyReLU)	(None, 6272)	0	batch_normalizat…
dense_2 (Dense)	(None, 1, 49)	2,499	embedding_1[0][0]
reshape_2 (Reshape)	(None, 7, 7, 128)	0	leaky_re_lu_2[0]…
reshape_1 (Reshape)	(None, 7, 7, 1)	0	dense_2[0][0]
concatenate_1 (Concatenate)	(None, 7, 7, 129)	0	reshape_2[0][0], reshape_1[0][0]
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 64)	206,400	concatenate_1[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 7, 7, 64)	256	conv2d_transpose…
leaky_re_lu_3 (LeakyReLU)	(None, 7, 7, 64)	0	batch_normalizat…
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 32)	51,200	leaky_re_lu_3[0]…
batch_normalizatio… (BatchNormalizatio…	(None, 14, 14, 32)	128	conv2d_transpose…
leaky_re_lu_4 (LeakyReLU)	(None, 14, 14, 32)	0	batch_normalizat…
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	801	leaky_re_lu_4[0]…

Total params: 914,072 (3.49 MB)
Trainable params: 901,336 (3.44 MB)
Non-trainable params: 12,736 (49.75 KB)

Step 5 : Loss function and optimizers

```
cross_entropy = keras.losses.BinaryCrossentropy()
generator_optimizer = keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
discriminator_optimizer = keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
```

```
print("Loss      : Binary cross-entropy")
print("Optimizer : Adam (learning rate = 0.0002, beta_1 = 0.5) for the generator and the
discriminator")
```

OUTPUT:

```
Loss      : Binary cross-entropy
Optimizer : Adam (learning rate = 0.0002, beta_1 = 0.5) for the generator and the discriminator
```

Step 6 : Conditional training step

```
@tf.function
def train_step(real_imgs, real_labels):
    batch = tf.shape(real_imgs)[0]
    noise = tf.random.normal((batch, LATENT_DIM))
    fake_labels = tf.random.uniform((batch, 1), 0, NUM_CLASSES, dtype=tf.int32)   # random class
to generate
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        fake_imgs = generator([noise, fake_labels], training=True)
        real_output = discriminator([real_imgs, real_labels], training=True)
        fake_output = discriminator([fake_imgs, fake_labels], training=True)
        d_real = cross_entropy(tf.ones_like(real_output), real_output)
        d_fake = cross_entropy(tf.zeros_like(fake_output), fake_output)
        d_loss = 0.5 * (d_real + d_fake)
        g_loss = cross_entropy(tf.ones_like(fake_output), fake_output)
    d_grads = disc_tape.gradient(d_loss, discriminator.trainable_variables)
    g_grads = gen_tape.gradient(g_loss, generator.trainable_variables)
    discriminator_optimizer.apply_gradients(zip(d_grads, discriminator.trainable_variables))
    generator_optimizer.apply_gradients(zip(g_grads, generator.trainable_variables))
    return d_real, d_fake, g_loss

print("Conditional training step defined.")
```

OUTPUT:

```
Conditional training step defined.
```

Step 7 : Train the Conditional GAN

```
EPOCHS = 20
BATCH_SIZE = 128
dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train.astype("int32").reshape(-1,
1))).shuffle(60000).batch(BATCH_SIZE, drop_remainder=True)
fixed_noise = np.random.normal(0, 1, (100, LATENT_DIM)).astype("float32")
fixed_labels = np.repeat(np.arange(10), 10).reshape(-1, 1).astype("int32")     # 10 samples for
every digit
history = {"d_real": [], "d_fake": [], "g": []}
snapshots = {}
SNAP_EPOCHS = sorted({1, EPOCHS // 4, EPOCHS // 2, EPOCHS})

for epoch in range(1, EPOCHS + 1):
    a, b, c = [], [], []
    for img_batch, lab_batch in dataset:
        d_real, d_fake, g_loss = train_step(img_batch, lab_batch)
        a.append(float(d_real)); b.append(float(d_fake)); c.append(float(g_loss))
    history["d_real"].append(np.mean(a)); history["d_fake"].append(np.mean(b));
history["g"].append(np.mean(c))
    if epoch in SNAP_EPOCHS:
        snapshots[epoch] = generator([fixed_noise, fixed_labels], training=False).numpy()
    if epoch == 1 or epoch % 2 == 0:
        print(f"Epoch {epoch:>2}/{EPOCHS}  d_loss(real)={history['d_real'][-1]:.3f}
d_loss(fake)={history['d_fake'][-1]:.3f}  g_loss={history['g'][-1]:.3f}")
generator.save("cgan_generator.keras")
print("Conditional GAN training completed. Generator saved.")
```

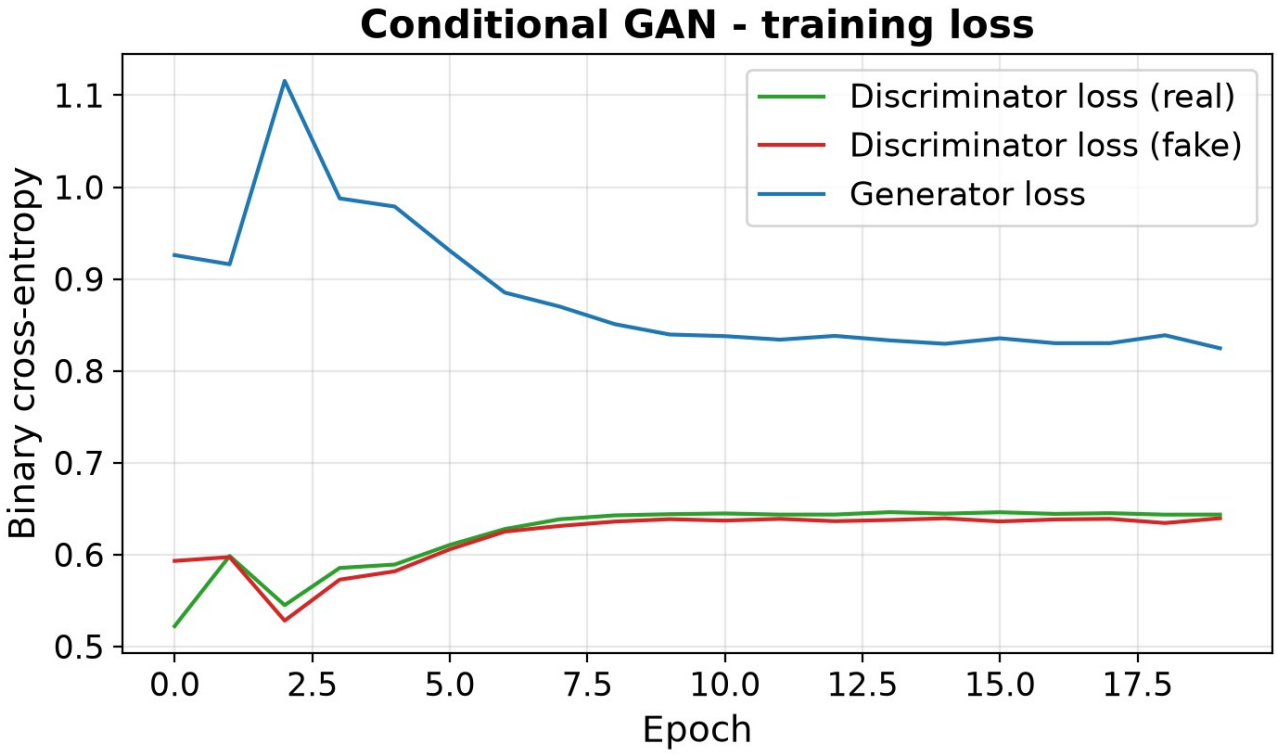
OUTPUT:

```
Epoch  1/20  d_loss(real)=0.522  d_loss(fake)=0.593  g_loss=0.926
Epoch  2/20  d_loss(real)=0.599  d_loss(fake)=0.597  g_loss=0.916
Epoch  4/20  d_loss(real)=0.585  d_loss(fake)=0.573  g_loss=0.988
Epoch  6/20  d_loss(real)=0.610  d_loss(fake)=0.606  g_loss=0.931
Epoch  8/20  d_loss(real)=0.638  d_loss(fake)=0.631  g_loss=0.870
Epoch 10/20  d_loss(real)=0.644  d_loss(fake)=0.639  g_loss=0.840
Epoch 12/20  d_loss(real)=0.644  d_loss(fake)=0.639  g_loss=0.834
Epoch 14/20  d_loss(real)=0.646  d_loss(fake)=0.638  g_loss=0.833
Epoch 16/20  d_loss(real)=0.646  d_loss(fake)=0.636  g_loss=0.835
Epoch 18/20  d_loss(real)=0.645  d_loss(fake)=0.639  g_loss=0.830
Epoch 20/20  d_loss(real)=0.644  d_loss(fake)=0.640  g_loss=0.825
Conditional GAN training completed. Generator saved.
```

Step 8 : Training curves

```
plt.figure(figsize=(8, 4.2))
plt.plot(history["d_real"], label="Discriminator loss (real)", color="#2ca02c")
plt.plot(history["d_fake"], label="Discriminator loss (fake)", color="#d62728")
plt.plot(history["g"], label="Generator loss", color="#1f77b4")
plt.title("Conditional GAN - training loss"); plt.xlabel("Epoch"); plt.ylabel("Binary cross-
entropy"); plt.legend(); plt.grid(alpha=.3)
plt.show()
```

OUTPUT:

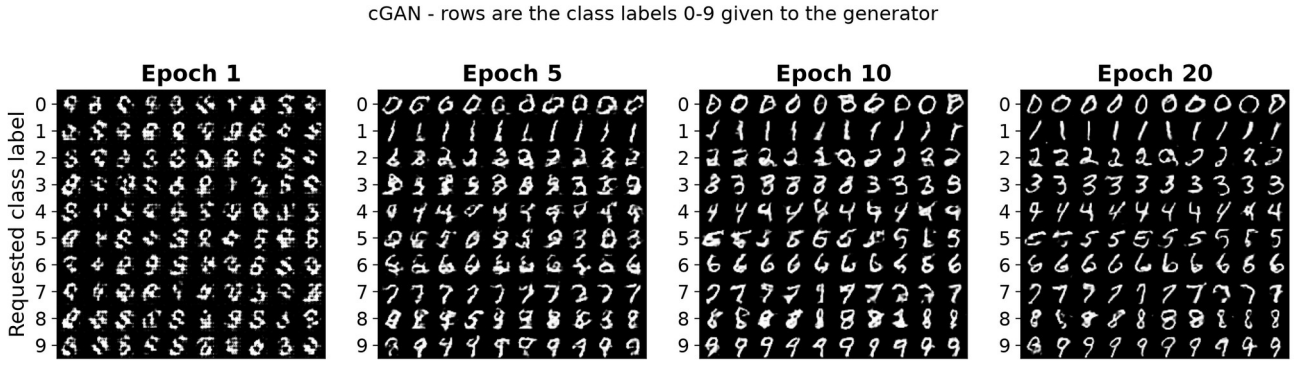


Step 9 : Class-controlled generation (progress)

```
fig, axes = plt.subplots(1, len(snapshots), figsize=(3.7 * len(snapshots), 4.2))
for ax, ep in zip(axes, snapshots):
    g = ((snapshots[ep][..., 0] + 1) / 2).reshape(10, 10, 28, 28).transpose(0, 2, 1,
3).reshape(280, 280)
    ax.imshow(g, cmap="gray"); ax.set_title(f"Epoch {ep}"); ax.set_xticks([]);
ax.set_yticks(np.arange(14, 280, 28)); ax.set_yticklabels(range(10))
axes[0].set_ylabel("Requested class label")
```

```
plt.suptitle("cGAN - rows are the class labels 0-9 given to the generator", fontsize=13)
plt.show()
```

OUTPUT:



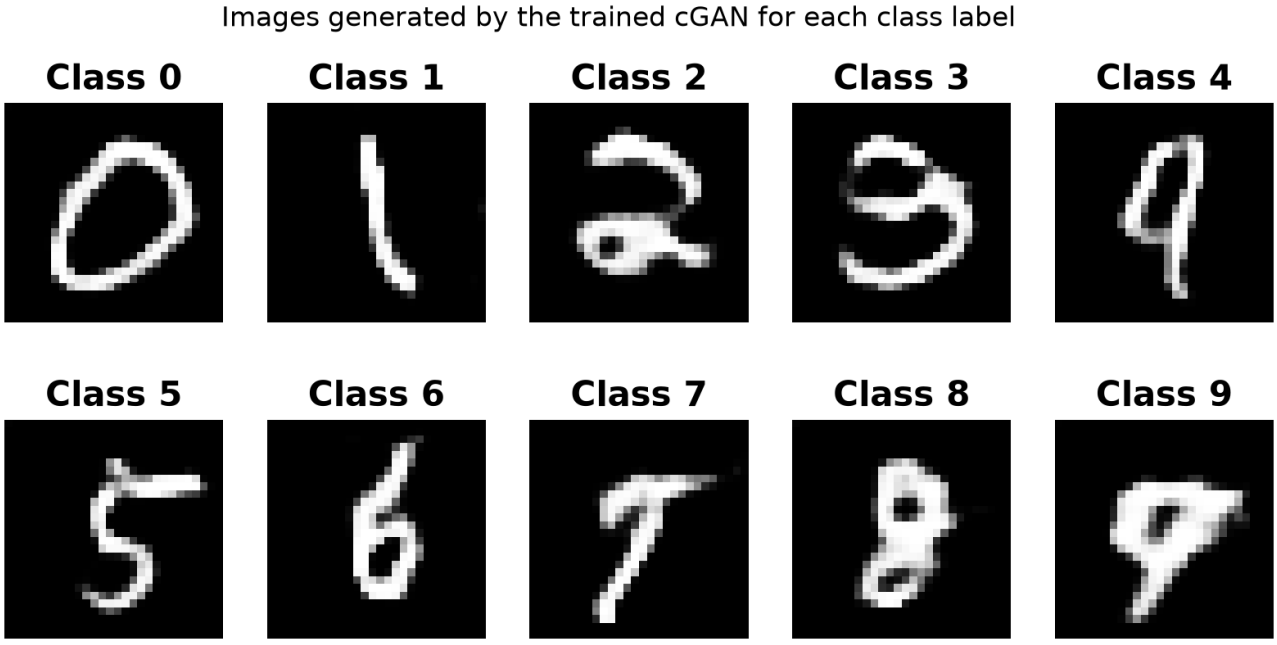
Step 10 : Generate digits for the requested labels

```
labels = np.arange(10).reshape(-1, 1).astype("int32")
noise = np.random.normal(0, 1, (10, LATENT_DIM)).astype("float32")
images = (generator([noise, labels], training=False).numpy() + 1) / 2
print("Requested labels :", labels.ravel().tolist())
print("Generated batch  :", images.shape)

fig, axes = plt.subplots(2, 5, figsize=(10, 4.6))
for ax, img, lab in zip(axes.ravel(), images, labels.ravel()):
    ax.imshow(img[:, :, 0], cmap="gray"); ax.set_title(f"Class {lab}"); ax.axis("off")
plt.suptitle("Images generated by the trained cGAN for each class label", fontsize=12)
plt.show()
```

OUTPUT:

```
Requested labels : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Generated batch  : (10, 28, 28, 1)
```



Step 11 : Verify that the generated digits obey the label

```
# a small CNN classifier trained on real MNIST is used as an independent judge
(xc, yc), (xt, yt) = keras.datasets.mnist.load_data()
```

```
xc = np.expand_dims(xc / 255.0, -1).astype("float32"); xt = np.expand_dims(xt / 255.0, -
1).astype("float32")
judge = keras.Sequential([keras.Input((28, 28, 1)), layers.Conv2D(32, 3, activation="relu"),
layers.MaxPooling2D(),
                          layers.Conv2D(64, 3, activation="relu"), layers.MaxPooling2D(),
layers.Flatten(),
                          layers.Dense(10, activation="softmax")])
judge.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
judge.fit(xc, yc, epochs=2, batch_size=128, verbose=2)
print("Judge accuracy on real MNIST test set : %.2f%%" % (100 * judge.evaluate(xt, yt, verbose=0)
[1]))

n = 2000
lab = np.random.randint(0, 10, (n, 1)).astype("int32"); z = np.random.normal(0, 1, (n,
LATENT_DIM)).astype("float32")
fake = (generator([z, lab], training=False).numpy() + 1) / 2
pred = judge.predict(fake, verbose=0).argmax(1)
print("Conditional accuracy (generated digit == requested label) : %.2f%%" % (100 * (pred ==
lab.ravel()).mean()))
for d in range(10):
    m = lab.ravel() == d
    print(f"  class {d}: {100 * (pred[m] == d).mean():6.2f}%   ({m.sum()} images)")
```

OUTPUT:

```
Epoch 1/2
469/469 - 10s - 21ms/step - accuracy: 0.9268 - loss: 0.2651
Epoch 2/2
469/469 - 8s - 18ms/step - accuracy: 0.9780 - loss: 0.0710
Judge accuracy on real MNIST test set : 98.26%
Conditional accuracy (generated digit == requested label) : 96.85%
  class 0:  98.10%   (210 images)
  class 1:  99.47%   (187 images)
  class 2:  97.09%   (206 images)
  class 3:  95.36%   (194 images)
  class 4:  93.55%   (186 images)
  class 5: 100.00%   (192 images)
  class 6:  99.53%   (213 images)
  class 7:  97.63%   (211 images)
  class 8:  93.75%   (208 images)
  class 9:  93.78%   (193 images)
```

RESULT:

Thus, the Conditional GAN was implemented successfully. The generator produced digit images of the class that was requested; an independent CNN classifier recognised 96.85 % of the generated images as the requested digit, which confirms the class-controlled generation.